

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

MASTER THESIS

Bc. Hugo Hrbáň

Interpreting and Steering Generative Protein Language Models

Department of Software Engineering

Supervisor of the master thesis: doc. RNDr. David Hoksza, Ph.D.

Study programme: Computer Science - Language
Technologies and Computational
Linguistics

Prague 2026

I declare that I carried out this master thesis on my own, and only with the cited sources, literature and other professional sources. During the preparation of this thesis, I used large language models to improve my writing style, readability of the text, formatting, and generate parts of the code. After using the aforementioned artificial intelligence tools, I thoroughly revised and modified the content as required and accept full responsibility for the resulting form of the thesis. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

I am grateful to David for consultations and for guiding me through the field bioinformatics, both in this thesis but also long before then. I am grateful to my friends and family for their constant support throughout my studies. Lastly, I would like to say a big thank you to Penda for always being there, for motivating me, and for making me finish writing this thesis.

Title: Interpreting and Steering Generative Protein Language Models

Author: Bc. Hugo Hrbáň

Department: Department of Software Engineering

Supervisor: doc. RNDr. David Hoksza, Ph.D., Department of Software Engineering

Abstract: Proteins are macromolecules essential to all living organisms. Protein language models (PLMs) trained on protein sequences are used in bioinformatics to predict their properties or to generate novel proteins. However, the inner workings of these large neural networks are not well understood. In this thesis, we build on the InterPLM project, which uses sparse autoencoders to interpret the features of a masked PLM. We extend this approach to a generative PLM and compare the interpretability of its features with those of the masked model. The work introduces new feature analysis methods, demonstrates their practicality, and extends the interactive web application for exploring features by concept or protein of interest. Finally, we use these features to steer the model's behavior during generation. This is achieved by clamping activations of selected features so as to induce the desired properties in the generated protein sequences. Analysis of the results shows that this kind of causal intervention is possible for a generative protein language model, as we demonstrate by generating disordered proteins and transit peptides.

Keywords: bioinformatics, protein language models, interpretability, activation steering

Název práce: Interpreting and Steering Generative Protein Language Models

Autor: Bc. Hugo Hrbáň

Ústav: Ústav formální a aplikované lingvistiky

Vedoucí diplomové práce: doc. RNDr. David Hoksza, Ph.D., Katedra softwarového inženýrství

Abstrakt: Proteiny jsou makromolekuly nezbytné pro všechny živé organismy. Proteinové jazykové modely (PLM) trénované na sekvencích proteinů se v bioinformatice používají k predikci vlastností proteinů nebo ke generování nových proteinů. Vnitřní fungování těchto velkých neuronových sítí však není dobře prozkoumáno. V této práci navazujeme na projekt InterPLM, který k interpretaci příznaků maskovaného PLM využívá řídké autoenkodéry. Tento přístup rozšiřujeme na generativní PLM a porovnáváme interpretovatelnost jeho příznaků s příznaky maskovaného modelu. Práce zavádí nové metody analýzy příznaků, demonstruje jejich praktickou využitelnost a rozšiřuje interaktivní webovou aplikaci pro prozkoumávání příznaků podle konceptu nebo proteinu zájmu. Získané příznaky nakonec využíváme k řízení chování modelu při generování. Toho dosahujeme fixací aktivací vybraných příznaků tak, aby generované proteinové sekvence měly požadované vlastnosti. Analýza výsledků ukazuje, že takový kauzální zásah je u generativního proteinového jazykového modelu možný, což dokládáme generováním neuspořádaných proteinů a tranzitních peptidů.

Klíčová slova: bioinformatika, proteinové jazykové modely, interpretabilita, aktivací řízení

Contents

Introduction	7
1 Biological Background	9
1.1 Proteins	9
1.1.1 Amino Acids	9
1.1.2 Protein Sequence and Structure	10
1.1.3 Functional Properties and Evolution	12
1.1.4 De novo Protein Design	14
1.2 Bioinformatical Databases and Tools	14
1.2.1 Databases	14
1.2.2 Tools	15
2 Machine Learning Background	17
2.1 Language Modeling	17
2.2 Neural Networks	17
2.2.1 Transformer Architecture	18
2.2.2 Training	19
2.3 Interpretability	21
2.3.1 Logit Lens	21
2.3.2 Probing Classifiers	22
2.3.3 Sparse Autoencoders	22
2.3.4 Activation Steering	23
2.4 ML in Bioinformatics	24
2.5 InterPLM	25
3 Methodology	27
3.1 Pipeline	27
3.2 Interpreting the Features	34
3.2.1 Per-concept Analysis	35
3.2.2 Per-protein Analysis	36
3.2.3 Dashboard	36
3.3 Activation Steering	38
4 Results	41
4.1 Training Sparse Autoencoders	41
4.2 Generative vs. Masked Language Models	43
4.3 Feature Case Studies	46
4.3.1 ATP Binding Site	46
4.3.2 Nucleophile Active Site	47
4.3.3 C2H2 Zinc Finger	48
4.4 Steering Experiments	49
4.4.1 Disorder and Compositional Bias	51
4.4.2 Transit Peptides	57
4.4.3 Domain and Motif Generation	59

5	Discussion	62
5.1	Limitations	62
5.2	Future Work	62
	Conclusion	64
	Bibliography	65
	List of Figures	71
	List of Tables	73
A	Attachments	74
A.1	Code	74
A.2	Changelog	74
A.3	Models	78

Introduction

Proteins carry out almost every function a living cell needs done, from catalyzing reactions to transporting molecules to recognizing and neutralizing pathogens, and all of this comes from a sequence of amino acids folding into a specific shape. For a long time, computational work on proteins was mostly about reading: given a sequence, predict its structure, its function, or which family it belongs to. Protein design turns this around. Instead of asking what a sequence does, we ask which sequence would do what we want, for example bind a specific molecule, catalyze a reaction that does not occur in nature, or replace a protein disabled by disease. This is known as *de novo* protein design, and it depends on having generative models that can propose new sequences rather than only classify existing ones.

A protein sequence is a string over an alphabet of 20 amino acids, so it can be treated as a kind of language, and this is exactly what protein language models (PLMs) do. Just as a language model learns to predict the next word in a sentence, a PLM is trained on large databases of protein sequences to predict a residue from the context, and in doing so it picks up regularities in how sequences are built. This idea carried over from NLP directly, following the transformer neural network architecture [1]. ESM-2 [2], for instance, is a masked language model, similar to BERT [3], trained to fill in residues that have been hidden from a sequence. ProGen2 [4], on the other hand, is a decoder-only, autoregressive model, closer to something like GPT [5], trained to predict the next residue given only the residues before it.

The problem with these models is that we do not really know how they arrive at their predictions. A PLM is only ever trained to predict a residue from its context, but somewhere inside its hidden layers it appears to represent much more than that, including structural and functional properties of the protein it is processing. This information is not laid out cleanly. A single neuron in the model’s hidden state typically responds to several unrelated concepts at once, a phenomenon called *superposition* [6], which makes the raw activations hard to interpret directly. Mechanistic interpretability is the field that tries to open up this black box, and one of the tools it uses is the sparse autoencoder (SAE), which re-encodes a model’s hidden activations into a much larger, sparse set of features, in the hope that each feature ends up corresponding to one interpretable concept instead of several tangled-up ones.

This approach was pioneered by Anthropic, who trained SAEs on the hidden activations of their Claude model and found individual features corresponding to concrete, human-interpretable concepts [7], [8]. InterPLM [9] applied the same idea to a protein language model for the first time, training SAEs on ESM-2 and showing that many of the resulting features align with known protein annotations, such as domains, binding sites, and secondary structure elements. This thesis follows directly from InterPLM, and asks whether the same approach also works for a generative protein language model, and whether interpreting these features lets us do more than just observe them.

Concretely, our work has three objectives, which move roughly from seeing what a model’s features represent, to making that easier to use, to actually controlling what the model generates.

- Our first objective is to replicate the InterPLM pipeline on ProGen2, a generative, decoder-only PLM, and to compare how interpretable its features are against the masked ESM-2 models used in the original publication.
- Our second objective is to extend the analysis tooling built around this pipeline. The original InterPLM dashboard lets a user browse features one at a time, but there was no direct way to start from a concept or a protein of interest and find the features related to it. We build two new modes of analysis that do exactly this, meant to make the process faster both for us and for future users of the pipeline.
- Our third and largest objective is activation steering. Once we have features that track a given property, we can ask whether we can also manipulate them to make the model generate that property on demand, rather than only detect it after the fact. This matters because it offers an alternative to finetuning a separate model for every desired property, which is expensive and needs labeled training data for each task. In NLP, an instruction-tuned model can simply be asked, in natural language, to produce text with some property. Generative PLMs typically have no such interface. The motivation behind activation steering is that we can train an SAE once on a given layer, and then, at inference time, steer whichever feature we found to be responsible for a desired property, without touching the model’s own weights. Previous work already demonstrated this kind of intervention for LLMs [10], and part of our own motivation here was curiosity about whether the same kind of intervention works at all for a generative protein language model. We show that it does on several tasks, and we also report the tasks where it did not work, along with our hypotheses for why.

The rest of this thesis is organized as follows:

- Chapter 1 covers the biological background, namely proteins, their structure and function, and the bioinformatics tools and databases we rely on throughout.
- Chapter 2 introduces the machine learning background, concretely language models and the interpretability techniques used to study them, and reviews related work.
- Chapter 3 describes the InterPLM pipeline in detail, including the improvements we made to it and the process of extending it to ProGen2 and adding new analysis methods.
- Chapter 4 presents our results. We first describe how we trained the SAEs, then compare the interpretability of the generative and masked PLMs, use the dashboard and our own analysis tools to find interesting features, and finally use those features to steer the model and report the results of our steering experiments.
- Chapter 5 discusses limitations of our approach and directions for future work.

1 Biological Background

In this chapter, we provide the biological background necessary to follow the work presented in this thesis. We describe proteins, their structure, and the different ways they can be represented computationally, either as sequences or as three-dimensional structures, and discuss what proteins do. We also introduce several databases and computational tools that we rely on throughout this work.

1.1 Proteins

Proteins play a role in virtually every biological process. They are found in all living organisms, across environments as extreme as deep-sea hydrothermal vents or Antarctic ice. Their functions are equally varied. Hemoglobin carries oxygen through the bloodstream. Antibodies recognize and neutralize pathogens. Enzymes catalyze chemical reactions with atomic precision, often accelerating them by many orders of magnitude compared to the uncatalyzed reaction. Ion channels in neurons open and close within milliseconds to propagate electrical signals. All of this functional diversity emerges from the chain of amino acids that folds into a specific three-dimensional shape.

Understanding how a protein achieves its function is not simple. A protein's function depends on its three-dimensional structure, and determining that structure experimentally requires techniques such as X-ray crystallography or cryo-electron microscopy, which are expensive, time-consuming, and not always feasible. Even when the structure is known, understanding exactly how it carries out its function, what other molecules it interacts with, and how it behaves inside a living cell remains difficult. On the other hand, "reading" the sequence of amino acids that make up a protein is much easier and therefore protein sequence databases are several orders of magnitude larger than databases of structures [11], [12]. The relationship between sequence and function is complex, and small changes to sequence can either abolish function entirely or have no effect at all. Beyond individual proteins, most sequences in public databases remain poorly annotated and their properties are not fully understood. Even though we may know the sequence, but not what the protein does, where it is found, or what role it ultimately plays [13].

Understanding the relationship between protein sequence and function more deeply could shed light on the causes of disease and how they can be cured, create enzymes for catalyzing novel reactions, or improve biocatalysis which focuses on manufacturing materials through reactions. Realizing this potential requires tools for studying, predicting, and generating proteins, both experimental and computational. This gap between what we know and what is possible is what motivates much of modern protein science, including the work presented in this thesis [13].

1.1.1 Amino Acids

Amino acids are small molecules. Each one has a carboxyl group ($-\text{COOH}$) and an amino group ($-\text{NH}_2$), connected through a central $\text{C}\alpha$ atom. Attached to

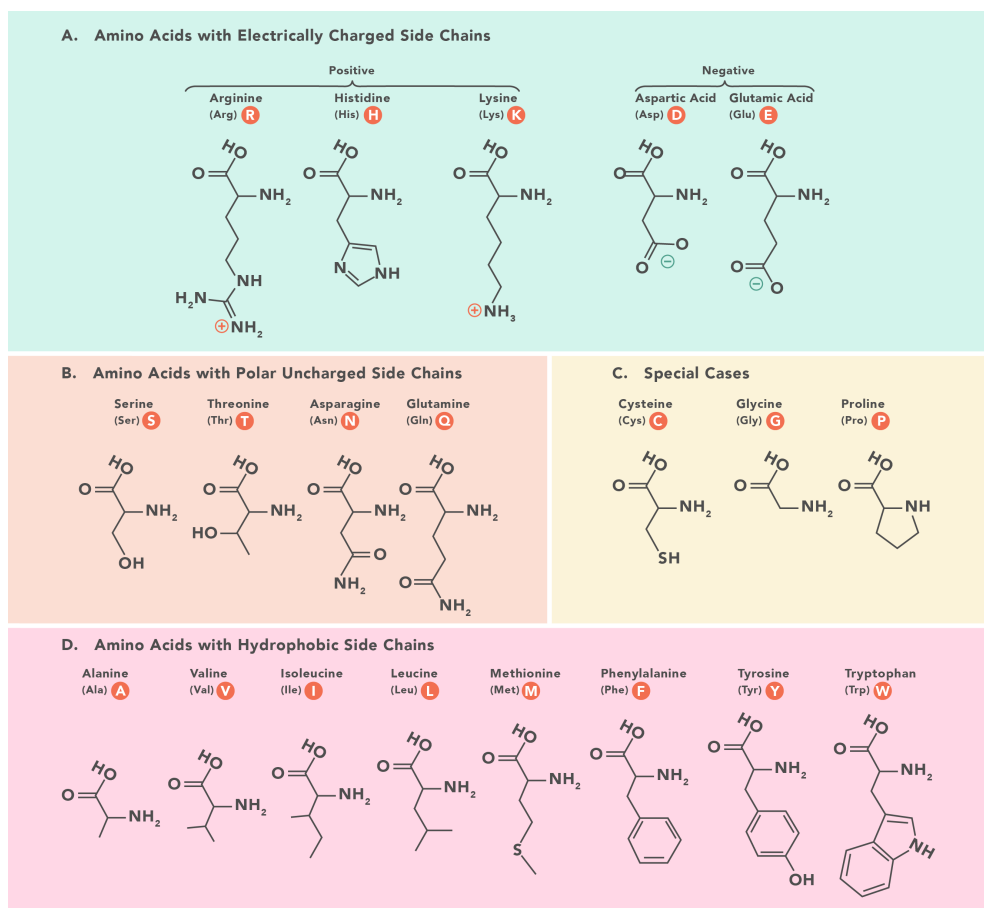


Figure 1.1 Table of standard 20 amino acids, including the one- and three-letter codes, grouped by physicochemical properties. Retrieved on 29.6.2026 from <https://cdn.technologynetworks.com/tn/images/body/aminoacids-pic3revised1574260662291.png>

the C_{α} is the side chain, which differs between amino acids. The nature of the side chain determines the chemical properties of the amino acid. It can be positively or negatively charged, hydrophobic or hydrophilic, aromatic, and so on. There are 20 standard amino acids encoded in the human genome, each represented by a single letter or a three letter code (for example, A/Ala for alanine, G/Gly for glycine, etc.) [13]. This one-letter notation is how protein sequences are represented computationally throughout this work. An overview of all 20 amino acids, including their codes, grouped by their chemical properties is shown in Figure 1.1.

1.1.2 Protein Sequence and Structure

DNA is the “blueprint” of all living things. It is essentially a very long string over an alphabet of four nucleotides, abbreviated as A, T, C and G. DNA is transcribed into RNA, which is then translated in the ribosome into a chain of amino acids. Each consecutive triplet of nucleotides, called a codon, encodes one amino acid. The amino and carboxyl groups of neighboring amino acids are joined by peptide bonds, forming long chains. Once an amino acid is incorporated into such a chain, its side chain is also referred to as the residue [14]. Figure 1.2, parts

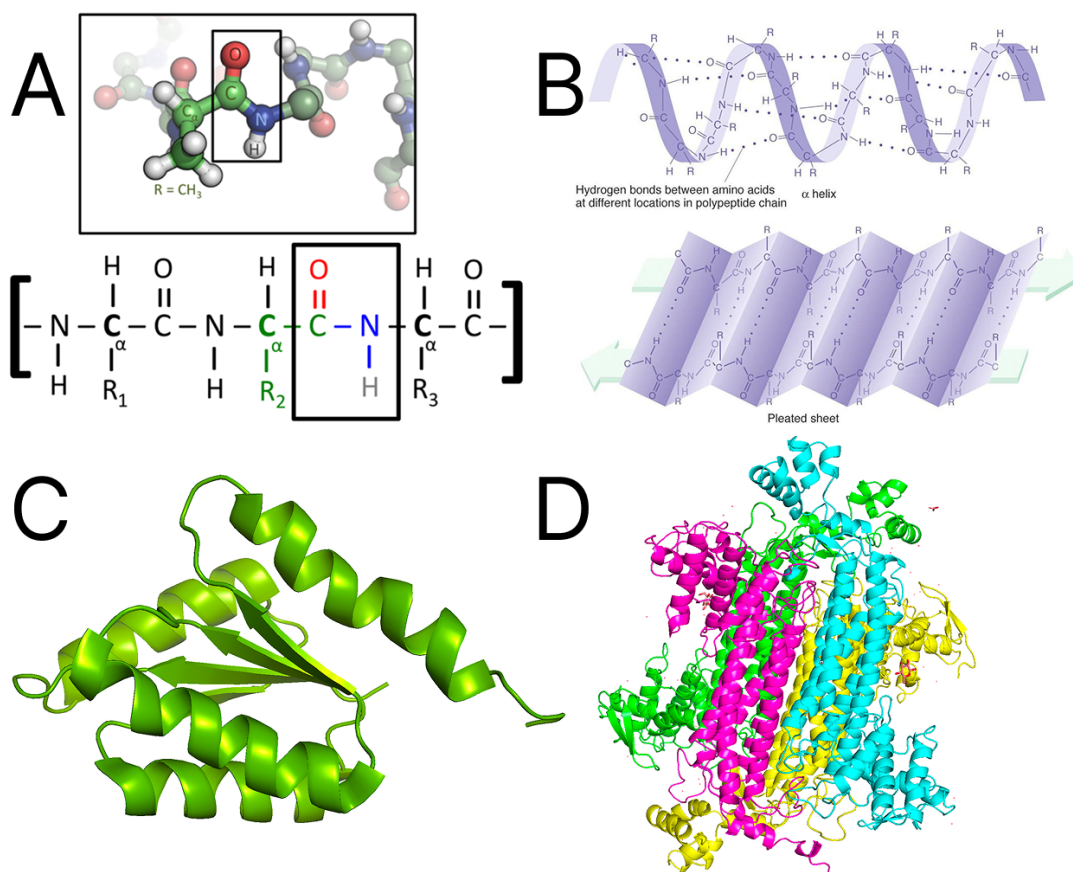


Figure 1.2 A. Chain of amino acids forming peptide bonds. retrieved on 9.7.2026 from <https://www.ebi.ac.uk/training/online/courses/biomacromolecular-structures/proteins/levels-of-protein-structure-primary/> B. Secondary structure elements, alpha helix and beta sheet. retrieved on 9.7.2026 from https://www.mun.ca/biology/scarr/MGA2_03-18b.html C. Ternary structure of a P-loop protein (PDB: 6C2U). D. Quaternary structure of ammonia lyase (PDB 1JSW), colored by chain, screenshots from PyMOL.

A illustrates this process alongside the resulting sequence representation.

A protein sequence folds spontaneously into a three-dimensional shape. When we refer to the structure of a protein, we typically mean this three-dimensional fold of a single polypeptide chain, also called its tertiary structure. The folding is driven largely by the tendency of hydrophobic residues to bury away from water, forming a hydrophobic core. Along the way, local regions of the backbone adopt regular, repeating arrangements, such as stretches of the chain coil into helices known as alpha helices, and other stretches form flat sheets known as beta sheets (collectively referred to as secondary structure elements), both stabilized by hydrogen bonds between backbone atoms. Two residues that are far apart in the sequence can end up in close contact in three-dimensional space after folding [15]. The sequence of a protein determines its structure, a principle established experimentally [16]. Predicting the tertiary structure from sequence alone was an open problem for decades, until AlphaFold [17] caused a major breakthrough in the field. Figure 1.2, parts B,C show an example protein structure alongside its secondary structural elements. In some cases a protein consists of multiple chains

that associate into an assembly, referred to as a quaternary structure, as shown in 1.2, part D.

1.1.3 Functional Properties and Evolution

Proteins perform their biological roles through specific, identifiable features encoded in their sequences: modular domains, disordered regions, targeting signals, and co-evolutionary patterns, among others. To understand where these features come from and why they are sequence-encoded, it helps to first understand how protein sequences evolve. We therefore begin with evolution and co-evolution before turning to the specific sequence features that appear throughout this work.

Evolution and Co-evolution

Proteins evolve because the DNA of organisms evolves as well. [18] Mutations can be insertions, deletions, or substitutions. A mutation can have essentially no effect on protein function, but sometimes even a single substitution can have severe consequences. In sickle cell disease, for example, one substitution in hemoglobin causes the protein to polymerize under low-oxygen conditions, deforming red blood cells and impairing oxygen delivery. In nature, over long timescales, proteins evolve through natural selection, same as species do. Variants that improve fitness tend to survive and spread, while harmful or dysfunctional variants are removed from the population. For example, directed evolution experiments replicate this process in the lab in order to improve enzyme function. Random mutations are introduced into a working enzyme, generating a library of variants that are then screened for how well they perform. The best-performing variants are selected for another round of mutation and screening. This approach has been experimentally shown to improve enzyme fitness [19].

Co-evolution refers to the phenomenon where multiple positions in a protein evolve together because they influence or depend on each other to maintain function. Over long evolutionary timescales, co-evolution and selection together give rise to protein families, which are groups of proteins that share a common evolutionary ancestor, similar sequences, and related functions. Within a family, certain positions are more conserved than others. The most conserved regions often correspond to functionally critical sites, and these recur across many different proteins as conserved domains and motifs. This is directly relevant to computational methods: protein language models trained on large sequence databases learn the statistical distribution of amino acids in proteins, implicitly capturing which positions co-vary and which are conserved, and in doing so they learn something about protein function.

Domains, motifs and regions

Proteins are not homogeneous along their length. Recurring structural and functional units called domains appear across many different proteins, and shorter conserved sequences called motifs mark specific functional sites. A domain can be identified, for example, by a motif, which is essentially the same as regular expression. Another way to identify domains in protein sequences is hidden Markov models [20], or position-specific scoring matrices (PSSM), which is a

matrix that defines how likely each amino acid is to appear at a specific position [21].

Intrinsic Disorder

Not all proteins fold into a well-defined, stable three-dimensional structure. Some proteins, or particular regions within them, are *intrinsically disordered*, meaning they do not adopt a single fixed conformation but instead exist as dynamic ensembles of many different conformations in solution. Such regions are referred to as intrinsically disordered regions (IDRs), and proteins that contain them are called intrinsically disordered proteins (IDPs). Disorder may be confined to a region of an otherwise structured protein, for example a flexible linker connecting two stable domains [22].

Disorder is not the same as dysfunction. Disordered regions are often functionally important precisely because of their flexibility. They can interact with many different binding partners, undergo post-translational modifications such as phosphorylation, and form transient complexes that a rigid structured domain could not. In some cases a disordered region will fold upon binding to its partner, a mechanism called coupled folding and binding [23].

Because disordered regions lack a stable structure, they cannot be characterized by standard structural methods such as X-ray crystallography. Instead, disorder may be inferred from sequence using computational predictors, which we describe in the following section 1.2.2 and use in this work to evaluate whether generated proteins contain disordered content.

Signal and Transit Peptides

Cells are highly compartmentalized. Many proteins are synthesized in the cytoplasm but need to function elsewhere: at the plasma membrane, outside the cell entirely, or inside organelles such as mitochondria or chloroplasts. The cell routes proteins to the correct location using short sequences located at the N-terminus (the beginning) of the protein, called targeting signals, or signal/target peptides [24].

Signal peptides direct proteins into the secretory pathway, which leads to the endoplasmic reticulum and ultimately to the plasma membrane or to secretion outside the cell. Transit peptides serve the analogous role for proteins destined for mitochondria or chloroplasts. In both cases, the targeting sequence is recognized by the import machinery at the target membrane, and then cleaved off by a specific protease. The mature protein that arrives at its destination does not retain it.

Targeting sequences share a few consistent properties. They are located at the N-terminus, they are typically short, on the order of 15 to 70 residues, and they are often intrinsically disordered [25], which may facilitate their recognition and threading by translocation complexes. Because they are short, well-defined, and experimentally validated, signal and transit peptides are one of the annotated features in public databases by dedicated prediction tools, which can predict their presence from sequence alone.

1.1.4 De novo Protein Design

De novo protein design, which means designing novel proteins completely from scratch, is one of the main reasons to build generative protein models. Applications include binder design and scaffold design for enzymes.

Related work includes protein language models such as ProGen2 [4] and ESM-2 [2], protein folding models such as AlphaFold [17], ESMFold [2], and Boltz [26], and generative structure-based methods such as RFdiffusion [27]. In the longer term, the field aims to cure diseases and engineer enzymes for reactions that do not yet exist in nature, among other goals.

1.2 Bioinformatical Databases and Tools

Computational methods are a key part of biological research. The number of known protein sequences and structures is growing exponentially [11], and we need computational tools to process and understand this data.

1.2.1 Databases

We use several bioinformatical databases in this work, so we provide a short description of each.

UniProt

UniProt is a widely used protein sequence database [11]. It also comes in a clustered variant called UniRef, in which sequences are clustered at 50%, 90%, or 100% sequence identity using MMseqs2 [28], meaning that within a given clustering level any two sequences share at most that fraction of identity after alignment.

Swiss-Prot

Swiss-Prot [29] is the reviewed subset of UniProtKB that contains annotated sequences. Human annotators create and verify the annotations, which include domains, binding sites, signal peptides, active sites, and more. This makes Swiss-Prot high quality, though considerably smaller than the full UniProt.

PROSITE

PROSITE [30] is a database of protein profiles and motifs, which describe functional or structural patterns in proteins rather than storing individual sequences. A motif in PROSITE is essentially the same as a regular expression in computer science. A profile is a matrix of dimensions $\text{length} \times 20$ known as a position-specific scoring matrix (PSSM), where each row stores position-specific scores over the 20 amino acids derived from the observed frequencies at that position. PROSITE reports the precision and recall of each entry when querying UniProt, which gives an indication of how reliably each pattern identifies the intended annotation.

Protein Data Bank

The Protein Data Bank (PDB) is the largest database of experimentally determined protein structures [12]. Each entry corresponds to a specific structure and is linked to the publication that solved it. As with Swiss-Prot, the PDB is high quality but considerably smaller than sequence-only databases such as UniProt.

For reference, at the time of writing (June 2026), the PDB contains 256,550 structures and UniProtKB contains nearly 150 million sequences, and its reviewed subset, Swiss-Prot, contains 575,503 sequences (release 2026_02).

AlphaFold Database

The AlphaFold Database contains predicted structures for the sequences in UniProtKB, all folded using AlphaFold2 [17].

1.2.2 Tools

In addition to databases, we rely on several bioinformatics tools throughout this work. We describe each of them briefly here.

MMseqs2

MMseqs2 (Many-against-Many sequence searching) is a tool for fast protein sequence search and clustering [28]. The key idea is to represent sequences using short k-mers and index them in a way that allows very fast candidate retrieval before any alignment is performed. This makes MMseqs2 orders of magnitude faster than classical tools such as BLAST [31] while achieving comparable sensitivity. We use it in this work for sequence clustering, which allows us to remove near-duplicate sequences from a dataset and evaluate how novel generated sequences are relative to known proteins.

SignalP and TargetP

SignalP [25] and TargetP [32] are tools for predicting targeting signals from sequence alone. SignalP predicts the presence of a signal peptide and the position of the cleavage site. TargetP extends this to other targeting signals, including mitochondrial and chloroplast transit peptides. Both tools are neural network-based and trained on experimentally verified sequences from UniProt. We use them in this work to evaluate whether generated sequences contain valid targeting signals.

MobiDB-lite

MobiDB-lite [33] is a lightweight predictor of intrinsic disorder. It combines several fast, sequence-based disorder predictors and returns a per-residue consensus disorder score. We use MobiDB-lite to measure the fraction of disordered residues in generated sequences, which allows us to assess whether a model has learned to generate disordered regions when prompted to do so. Within each region predicted as disordered, the model also outputs regions of certain types of compositional

bias, for example low complexity characterized by repetitive motifs, polar residue bias containing higher frequency of hydrophilic residues, or residue-specific bias, such as glycine-rich regions.

ps_scan

ps_scan is a command-line tool distributed with the PROSITE database [30] that scans protein sequences against a particular profile/pattern or against the full library of PROSITE patterns and profiles. For each sequence, it reports which PROSITE entries match, at what positions, and with what score. We use it to check whether generated sequences contain recognizable functional domains or motifs.

2 Machine Learning Background

This chapter introduces the machine learning background for concepts used in this thesis. We start with language modeling and the neural network architecture behind it, the transformer, since every model we study and every intervention we make later operates on this architecture. We then turn to interpretability, in particular sparse autoencoders, and to the activation steering experiments on large language models that motivated our own experiments on protein language models. Finally we introduce InterPLM [9], the paper at the center of this thesis, together with other related work on interpreting protein language models.

2.1 Language Modeling

Language modeling is the task of estimating the probability of a sequence of tokens $W = (w_1, \dots, w_n)$. By the chain rule of probability, this joint probability factorizes into a product of conditionals,

$$P(W) = P(w_1) P(w_2 | w_1) \cdots P(w_n | w_{<n}) = \prod_{i=1}^n P(w_i | w_{<i}),$$

so that predicting a whole sequence reduces to repeatedly predicting one token from the tokens before it. What counts as a token depends on the model. It can be a character, a whole word, or, more commonly, a subword unit produced by a tokenizer. In protein language models, a token is typically a single amino acid, which gives a vocabulary of only 20 symbols plus a handful of utility tokens, much smaller than the vocabularies of natural language models, which commonly exceed 100,000 tokens.

This factorization only assumes that a token depends on the tokens before it. It says nothing about how that conditional distribution $P(w_i | w_{<i})$ is actually computed in practice, which is the job of the neural network described in the next section. Framed this way, language modeling is naturally generative. Once we have learned $P(w_i | w_{<i})$, we can sample a token from it, append it to the context, and repeat, recomputing the distribution at every step. This is called autoregressive generation, and it is how large language models produce text, whether as chatbots, coding assistants, or, as in this thesis, generators of protein sequences.

2.2 Neural Networks

At its core, a neural network is a parameterized function built out of neurons arranged in layers. A single neuron takes a weighted sum of its inputs, adds a bias term, and passes the result through a nonlinear activation function φ ,

$$y = \varphi \left(\sum_{i=1}^n w_i x_i + b \right) = \varphi(\mathbf{w}^\top \mathbf{x} + b),$$

where $\mathbf{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$ are learned parameters. The nonlinearity φ is essential, because without it, stacking several linear layers would be mathematically equivalent to a single linear map. Common choices for φ include the rectified linear unit, $\text{ReLU}(x) = \max(0, x)$, or smoother variants such as GELU.

Arranging neurons into layers and stacking several of them gives a multilayer perceptron (MLP), where each layer computes

$$\mathbf{h}^{(l)} = \varphi \left(W^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

with $\mathbf{h}^{(0)}$ the input to the network and $W^{(l)}, \mathbf{b}^{(l)}$ the weight matrix and bias of layer l . The output of the last layer is used directly for regression, or, after a softmax, as a probability distribution over a fixed set of classes. Softmax exponentiates every entry of a vector and normalizes the result to sum to one, $\text{softmax}(\mathbf{z})_i = e^{z_i} / \sum_{j=1}^K e^{z_j}$, turning any vector of real numbers into a valid probability distribution over its K entries. In language modeling that fixed set is the vocabulary. Training a network then means searching for values of every $W^{(l)}$ and $\mathbf{b}^{(l)}$ that make its output match some desired behavior.

An MLP on its own has no notion of order or of variable-length input. It expects a fixed-size vector, so it cannot naturally represent a sequence of arbitrary length, let alone let one token influence how another is represented. The transformer, described next, is the architecture that solves this, and it underlies every model used in this thesis.

2.2.1 Transformer Architecture

In recent years, the transformer architecture [1] has become the de-facto standard across many domains, including natural language and, as used throughout this thesis, protein sequences. It was originally introduced for machine translation, with an encoder that reads the input sentence and a decoder that generates the translation one token at a time. Earlier models of this kind used recurrent networks, which carry a single hidden state through the sequence one token at a time, so the entire input has to be compressed into that one state, making it hard to preserve dependencies between distant tokens. The transformer fixes this issue with attention, a mechanism that lets every token in a sequence look directly at every other token, regardless of how far apart they are.

Before any attention is computed, the input tokens first have to become vectors. An embedding layer is a lookup table $E \in \mathbb{R}^{|V| \times d}$ assigns each of the $|V|$ vocabulary items a learned d -dimensional vector, where d is the model’s hidden dimension. Attention itself has no sense of order. It treats its input as an unordered set of vectors, so permuting the input tokens would not change the output at all unless we add this information back in somehow. This is the role of positional encoding. ProGen2, the model used throughout this work, encodes position with rotary positional embeddings (RoPE) [34], which rotate the query and key vectors described below by an angle that depends on their position in the sequence, rather than adding a separate positional vector to each token.

Given the embedded, position-aware input X , a single attention head projects every token into a query, key, and value vector through learned linear maps, $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and combines them as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V,$$

where d_k is the dimension of the key vectors. The term $QK^\top$ measures how relevant every other token is to a given token, the softmax turns these relevance

scores into weights that sum to one, and the output is the corresponding weighted average of the value vectors. In practice, several such heads run in parallel, each with its own set of projections, and their outputs are concatenated and projected back down to the model dimension,

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O,$$

$$\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i),$$

so that different heads are free to specialize in different kinds of relationships between tokens.

A transformer layer wraps multi-head attention and a small MLP, also called the feed-forward sublayer, in residual connections, so that each sublayer only has to learn a modification to add on top of its input rather than a full replacement of it. If we denote as x_l the input to layer l and applying layer normalization LN before each sublayer, a placement known as pre-norm and used by most modern transformers, the forward pass computed by each layer is

$$x'_l = x_l + \text{Attention}(\text{LN}(x_l)), \quad x_{l+1} = x'_l + \text{MLP}(\text{LN}(x'_l)).$$

After the final layer, one more normalization and a linear projection map the model embeddings into vocabulary-sized logits, and a softmax turns these into the probability distribution over the next token. Figure 2.1 shows the architecture of the transformer model (encoder-decoder).

The encoder and decoder differ only in how attention is masked. In the encoder, a token attends to every other token in the sequence, in both directions. In the decoder, generation has to proceed left to right, so a token may only attend to itself and the tokens before it. This is enforced with a lower triangular mask that sets attention scores to $-\infty$ for any token to the right, before the softmax is applied.

Although the encoder and decoder were originally introduced together for translation, they can also be used separately. Encoder-only models are trained by masking a fraction of the input tokens and predicting them from their surrounding, bidirectional context, with BERT [3] as the canonical example in natural language. Decoder-only models instead use the causal, next-token objective described earlier, predicting each token only from what precedes it. The two protein language models compared in this thesis are instances of each type. ESM-2 [2] is encoder-only and trained with the masked LM objective, while ProGen2 [4] is decoder-only and autoregressive, which is also why only ProGen2 is suitable for generation and activation steering.

2.2.2 Training

Neural networks are trained with gradient-based optimization. We run the forward pass described above on a batch of training examples, compare its outputs to the desired targets with a loss function $\mathcal{L}$, and adjust every parameter θ of the network to minimize it. The direction of adjustment is given by the gradient $\nabla_{\theta}\mathcal{L}$, which backpropagation computes for every parameter in the network, by repeatedly applying the chain rule of derivatives from the loss backward through

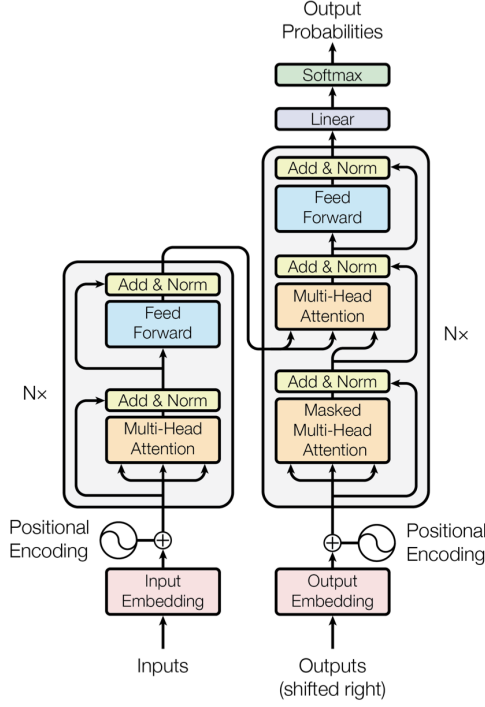


Figure 2.1 Architecture of the transformer model [1], showing both the encoder and decoder

the layers. The simplest update rule, gradient descent, then takes a small step against the gradient,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L},$$

where η is the learning rate. In practice, the gradient is computed on a small batch of examples rather than the whole dataset at once, known as mini-batch, or stochastic gradient descent. We repeat this update over the whole training set, split into batches, for many steps, while periodically checking the loss on held-out data the network was not trained on, to see whether it is actually generalizing rather than just memorizing what it has seen.

This whole procedure needs a target output for every training example, i.e. labels, which makes it supervised learning. Labels are often expensive to obtain in practice, since, for example, a dataset of translated sentence pairs or of images annotated with a text description both require manual work to produce. Language models use self-supervised learning, which means that the “label” at a given position is simply the token that already occurs there in the raw text, so no manual annotation is needed and the amount of available training data is limited only by how much text can be gathered. For an encoder-only model, this means predicting the tokens that were randomly masked out. For a decoder-only model, it means predicting every next token in the sequence, which gives as many training examples as there are tokens in the corpus.

The loss function used to train language models of both kinds is cross-entropy. At a given position, the model outputs a predicted distribution $\hat{p} \in \mathbb{R}^{|V|}$ over the vocabulary, while the target is the true next (or masked) token, represented as a

one-hot vector $p \in \mathbb{R}^{|V|}$. Cross-entropy is defined as

$$\mathcal{L}_{\text{CE}} = H(p, \hat{p}) = - \sum_{v=1}^{|V|} p_v \log \hat{p}_v.$$

Since p is one-hot, with $p_v = 1$ only for the true token v^* and zero everywhere else, every term in the sum but one vanishes, and the loss reduces to $\mathcal{L}_{\text{CE}} = -\log \hat{p}_{v^*}$, the negative log-probability the model assigned to the token that actually occurred. This quantity is small when the model is confident and correct, and grows without bound as the probability it assigns to the correct token goes to zero.

2.3 Interpretability

So far, we have described how a transformer computes its output, but not what happens to the input along the way. Interpretability is the field that tries to answer this, opening up a trained neural network to understand what it represents internally and why it produces the outputs it does. Rather than judging a network only by its outputs, interpretability studies its actual internal structures, the individual components, the transformations of the input vector and the computations connecting them, in much the same spirit as reverse engineering an unfamiliar piece of compiled software back into readable, human-understandable code [35]. Apart from only trying to understand the blackboxes which are neural networks, this also has some practical uses. If we can identify what a model represents, we can also try to causally intervene on it, changing a specific property of its output without retraining it at all. For example, for LLMs, this can be used to mitigate hallucinations of harmful behavior.

Much of this work is published under the label “mechanistic interpretability” (MI). The term itself is used inconsistently across the field, sometimes requiring a full causal account of a model’s computation and sometimes describing any inspection of its internals at all [36]. We use “interpretability” throughout this work as defined in the second way.

The obstacle to doing any of this is that a model’s internal activations are not laid out in a way that maps cleanly onto human concepts. A single neuron often responds to several unrelated things at once, a property called polysemanticity. In the vision model Inception v1, one neuron fires on both cat faces and the fronts of cars. In a small 1-layer language model from Bricken et. al [7], a single neuron fires on a mixture of academic citations, English dialogue, HTTP requests, and Korean text. Reading a model’s hidden state one neuron at a time therefore does not tell us much on its own. We describe a few simple tools for inspecting the model’s hidden states.

2.3.1 Logit Lens

One way to look inside a model without decomposing its neurons at all is the logit lens. Every transformer layer reads from and writes back into a shared sequence of vectors that runs through the whole network (input/output of each transformer layer), one per token position, which is referred to as the *residual stream* in many MI publications. It is only ever read out, at the very end, by a

final normalization and the unembedding matrix that turn the last layer’s residual stream into logits, from the model dimension d to the vocabulary dimension $|V|$. Nothing stops us from applying that same readout to an earlier layer’s residual stream x_l instead of x_L , giving a distribution over the vocabulary that reflects the model’s “current guess” at layer l rather than its final one [37]. This needs no additional training and is essentially free to compute, but it only ever tells us about the next-token distribution the model is converging toward, which can be useful for determining steps in which the model is uncertain in its prediction. However, it cannot say anything about a concept, such as the sentiment or topic of a passage, that is not itself a vocabulary item.

2.3.2 Probing Classifiers

A more flexible option is a probing classifier [38]. Given a concept we already have in mind, for example whether a sentence is written in the past or present tense, we label a set of activations with it and train a small classifier, often just a single linear layer, to predict the label from the activation vector [39].

A probe that decodes a concept well is evidence for what is called the linear representation hypothesis, the idea that high-level concepts correspond to directions in a model’s activation space. The classic illustration comes from word embeddings, where simple arithmetic on the vectors themselves tracks the underlying concepts, most famously $\text{vec}(\text{king}) - \text{vec}(\text{man}) + \text{vec}(\text{woman}) \approx \text{vec}(\text{queen})$, showing that the model learned a single “gender” direction and a single “royalty” direction, which can be added and subtracted independently of everything else in the vector [40]. The linear representation hypothesis generalizes this observation to the internal activations of much larger models [41].

Probing is cheap and does not require touching the model’s weights, but it needs labeled examples for every concept we want to check, one at a time, and a successful probe only shows that a concept is *present* in the activations. It does not show that the model actually uses that concept to produce its output, which is a different, causal question [39].

2.3.3 Sparse Autoencoders

Probing requires having a concept in mind and labels for it, and the polysemantic neuron described at the start of this section already suggests there may be far more concepts packed into a model than its dimension. Superposition is the hypothesis that explains this. It says that a layer of D neurons can represent far more than D concepts, or features, by giving each one its own direction in activation space rather than its own dimension [42]. Because there are then more directions than dimensions, they cannot all be orthogonal, so a single neuron ends up entangled in several unrelated features at once. This is precisely the polysemanticity described above, and superposition is the reason it happens. It also reframes what we are looking for. Rather than searching for the one direction that encodes a concept we already know about, we would like to recover a model’s entire feature basis without knowing in advance what any of the features are.

A sparse autoencoder (SAE) is trained to do exactly this, in an unsupervised way. It takes an activation vector $\mathbf{x} \in \mathbb{R}^D$, in our case a single token’s position in

the residual stream, and re-encodes it into a much larger set of feature activations $\mathbf{f} \in \mathbb{R}^F$ with $F \gg D$, using one linear layer followed by a ReLU, then reconstructs the original activation with a linear decoder,

$$\mathbf{f} = \text{ReLU}(W^{\text{enc}}\mathbf{x} + \mathbf{b}^{\text{enc}}), \quad \hat{\mathbf{x}} = W^{\text{dec}}\mathbf{f} + \mathbf{b}^{\text{dec}}.$$

Training minimizes a combination of the reconstruction error and a penalty S on the feature activations that encourages sparsity, so that only a few of the F features end up active for any single input,

$$\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda S(\mathbf{f}).$$

The original choice is $S(\mathbf{f}) = \|\mathbf{f}\|_1 = \sum_i |f_i|$ [7], which is easy to optimize with gradient descent but is really only a proxy for what we actually want, which is to directly limit how many features are active at all. As a measure of sparsity, we use L_0 , but it is not differentiable, so it cannot be used as a training penalty directly. TopK SAEs fix the problem by not using a penalty on $\mathbf{f}$ at all. Instead, the encoder keeps only the k largest feature activations for a given input and zeroes out the rest [43], which fixes the sparsity of every single reconstruction exactly, rather than only encouraging it on average through λ . This thesis uses a further refinement of this idea, BatchTopK [44], which relaxes the constraint from exactly k active features per token to nk active features across a whole training batch of n tokens at once. Some tokens can then use more active features than others while the average sparsity across the batch stays fixed at k , which in practice reconstructs activations more faithfully than a rigid per-token cutoff [44]. Since inference happens without a batch to share a threshold across, a single global activation threshold estimated from training statistics takes its place once training is done [44].

This general approach was first demonstrated on a small, one-layer transformer [7], and later scaled to a production-sized model, Claude 3 Sonnet, where the resulting features remained interpretable and included abstract, multilingual, even multimodal concepts [10].

2.3.4 Activation Steering

Clamping a feature’s activation to a certain value does not just correlate with a concept, it may in some cases actually cause the model to behave as though that concept were present. This is called *activation steering*, or *activation engineering*, a family of techniques that intervene directly on a model’s activations during the forward pass at inference time, changing what it generates without touching any of its weights or running any additional training [45], [10].

The general recipe does not require an SAE. Given a pair of prompts that differ mainly along one dimension of interest, for example one ending “I talk about weddings constantly” and another ending “I do not talk about weddings constantly,” we record the model’s activations on each and take their difference as a *steering vector* for that dimension [45]. Adding a multiple of this vector into the residual stream during generation shifts the output along the same dimension, often from as little as a single pair of contrasting examples and no gradient-based optimization at all.

Templeton et. al [10] demonstrate how SAE features can be used to achieve the same goal. Clamping the “Golden Gate Bridge feature” found in Claude 3 Sonnet’s SAE to ten times its maximum observed activation is enough to make the model start describing itself as the Golden Gate Bridge, and mention the bridge in practically any response. What makes this interesting is not just that it works, but that the effect matches the feature’s own interpretation, which is exactly the causal evidence a purely correlational tool like a probe cannot give us.

For an SAE specifically, steering has a clean mechanistic description. An SAE is trained to reconstruct an activation $\mathbf{x}$ from its feature activations $\mathbf{f}$, but that reconstruction $\hat{\mathbf{x}}$ is not perfect, so it is useful to write the original activation as the reconstruction plus whatever the SAE failed to capture,

$$\mathbf{x} = \text{SAE}(\mathbf{x}) + \text{err}(\mathbf{x}), \quad \text{where } \text{SAE}(\mathbf{x}) = \hat{\mathbf{x}} \text{ and } \text{err}(\mathbf{x}) = \mathbf{x} - \hat{\mathbf{x}}.$$

To steer feature i , we replace its activation f_i with a chosen value c , leave every other feature and the error term untouched, and reconstruct,

$$\tilde{\mathbf{x}} = W^{dec}\tilde{\mathbf{f}} + \mathbf{b}^{dec} + \text{err}(\mathbf{x}), \quad \tilde{f}_i = c, \quad \tilde{f}_j = f_j \text{ for } j \neq i,$$

and continue the forward pass through the remaining layers using $\tilde{\mathbf{x}}$ instead of $\mathbf{x}$, at every token position where we want the intervention active. Setting c to zero ablates the feature. Setting it far above the feature’s largest activation observed during training pushes the model to behave as if the corresponding concept were present [10].

2.4 ML in Bioinformatics

Protein language models are one instance of a much broader shift toward applying deep learning directly to biological data, largely by drawing inspiration from architectures and training objectives developed in the NLP field, but also new task-specific model architectures have been developed. The ESM family [2] and ProGen [4] are two threads of this, masked and generative respectively, as described above.

Once trained, these models are rarely used only for the objective they were trained on. Their embeddings are commonly used for downstream tasks such as predicting a protein’s function or enzyme class, whether a given mutation is stabilizing or destabilizing, or where in the cell a protein ends up. The general approach for this is to freeze the pretrained model, or fine-tune it lightly, and train a small supervised head on top of its embeddings for the task at hand. This kind of lightweight classifier is a way to get a good predictor without training a task-specific model from scratch, the same way BERT [3] and other pretrained language models are reused for downstream language processing tasks.

Language models are not the only place deep learning has reshaped bioinformatics. AlphaFold [17] was a major breakthrough in the decades-old problem of predicting a protein’s three-dimensional structure directly from its sequence to near-experimental accuracy, and generative diffusion models such as RFDiffusion [27] now design entirely new protein structures and sequences for a target function, going one step further than the sequence-level generation this thesis studies. These tools are why the 2024 Nobel Prize in Chemistry was awarded for computational protein structure prediction and protein design.

2.5 InterPLM

InterPLM [9] is the starting point for this thesis. They trained sparse autoencoders on ESM-2 [2] embeddings, across all six layers of the small, 8-million-parameter model and six layers of the much larger 650-million-parameter one, and show that the resulting features line up with known Swiss-Prot concepts, such as active sites, structural motifs, and functional domains, far more often than the raw neurons of either model do. This gap between neurons and SAE features holds at every layer and both model scales, and the larger model’s features also cover substantially more distinct biological concepts than the smaller one’s, which the authors take as evidence that scaling a PLM up does not remove superposition, it just gives the model more room to pack additional concepts into the same entangled neurons.

Since a single SAE produces thousands of features per layer, checking each one by hand against known biology does not scale. The authors address this with an automated interpretation pipeline built around a large language model, Claude 3.5 Sonnet. For a given feature, the LLM is shown its most highly activating proteins together with their Swiss-Prot annotations and asked to write a short natural-language description of when the feature fires. To check whether that description is actually any good, rather than just plausible, the same description is then used, on a held-out set of proteins the LLM has not seen, to predict how strongly the feature should activate on each one, and the predicted and true activation values are compared to give the description a quality score. Across more than a thousand features this gave a median correlation of 0.72 between predicted and true activation, providing evidence that the descriptions capture a real pattern rather than just fitting the examples they were written from.

The paper puts this framework to a few more concrete uses. First, cases where a feature fires strongly on a protein that carries no matching Swiss-Prot annotation often turn out to flag gaps in the annotations rather than errors in the model, for example a conserved Nudix-box motif, a peptidase S1 domain, or a glycosyltransferase binding site, each missing from the relevant UniProt entry despite the feature’s activation pattern, and independent structural evidence, clearly indicating it. The authors also cluster features by their SAE decoder vectors and visualize the result with UMAP, finding that features grouped this way tend to share a Swiss-Prot concept, such as kinase binding sites or TBDR beta-barrels. As a brief proof of concept on the side, they also show that clamping a feature tracking collagen’s periodic glycine repeats raises the predicted probability of glycine at other, still-masked positions following the same pattern. This is a narrow result though, since ESM-2 is masked and encoder-only and so only fills in already-masked positions rather than steering open-ended generation.

Related work

Several other publications have applied related tools to protein language models, either purely for interpretability or with an eye toward steering.

On the interpretability side, PLM-eXplain [46] takes a different route than an SAE, splitting an ESM2 embedding space into an interpretable biochemical subspace, built from properties such as secondary structure and hydrophathy, and a residual subspace that preserves predictive accuracy on downstream classification

tasks. ProtSAE [47] instead keeps the SAE architecture but guides its training semantically, incorporating domain knowledge to help disentangle the resulting features. Another work [48] also trains SAEs on ESM-2’s residual stream, having a large overlap with InterPLM, as they acknowledge. Additionally, they fit linear probes on the resulting features to recover known sequence determinants such as thermostability and subcellular localization.

On the steering side, one work [49] uses the same activation addition approach described above [45], computing a steering vector from the contrast between thermostable and non-thermostable sequences. It applies this iteratively: at each step, the sequence positions that most hurt the current sequence’s predicted stability are identified, masked, and resampled while steering with the vector, gradually pushing the whole sequence toward higher stability. Another related study [50] trains SAEs on ESM-2 and, after identifying features tied to properties such as transmembrane regions and binding sites, uses those same features to steer generation toward a target domain such as a zinc finger. Their generation, however, is performed from a fully masked sequence and unmasked using ESM-2, which was only trained on sequences with up to 15% masked tokens. Closest to our own approach, another study [51] trains SAEs on the decoder-only enzyme model ZymCTRL and steers sequence generation toward higher predicted enzymatic activity by clamping and ablating SAE features identified through sparse logistic regression, also introducing an alignment-aware variant of the technique called MSA steering.

3 Methodology

In this chapter, we describe the technical details of replicating the InterPLM pipeline [9] on ProGen2 [4], which is a generative protein language model. Beyond reusing the existing pipeline code, we made several improvements to its computational efficiency and added new features. Unless otherwise stated, the code and scripts described here were originally implemented by the InterPLM authors. The original code is at <https://github.com/ElanaPearl/InterPLM> and our fork with improvements and additional features is at <https://github.com/hugohrban/InterPLM>. A comprehensive changelog of our contributions is in the appendix A.2.

3.1 Pipeline

Firstly, we describe the InterPLM pipeline, including data preprocessing, training and finally interpreting the features from sparse autoencoders (SAEs). Apart from describing the theory and technicalities of the existing pipeline, we also report our journey of replicating it and extending it for application to ProGen2 [4]. We show a high-level schema of the entire interpretability pipeline in figure 3.1.

Gathering hidden layer embeddings

The input data consists of protein sequences from the UniRef50 dataset, which is UniProt [11] clustered at 50% sequence identity, meaning that any two sequences in the dataset are at most 50% identical after alignment. The pipeline includes a script for selecting a random subset of UniRef50 and another for splitting the data into fixed-size shards. After sharding, we can optionally reserve a few shards as a test split, with the remainder used for training. The test shards are later used for fidelity evaluation after SAE training, to assess reconstructability. Each shard is a FASTA file containing a fixed number of sequences, with the last shard potentially containing fewer.

After sharding, we extract PLM embeddings, which serve as training and test data for the SAE. The InterPLM library provides an **Embedder** class, which is a wrapper around a protein language model that extracts embeddings from a specific hidden layer or from multiple layers. The input can be a sequence string or a FASTA file. The embeddings are saved as **.pt** files, and the metadata, including the model checkpoint, embedding dimension, layer index, and number of tokens, is saved in YAML format.

For our use case, we added a **ProGenEmbedder** class that inherits from the base **BaseEmbedder**. It is largely the same as the existing ESM embedder, with one notable difference. When processing a sequence with ProGen2, we prepend the 1 token and append the 2 token, which denote the N- and C-terminus respectively, and we tokenize using ProGen2’s custom tokenizer. During the forward pass, the model computes embeddings for all tokens including the terminus tokens. We do not store the terminus token embeddings, both for compatibility with the downstream pipeline and to ensure the number of stored embeddings matches the

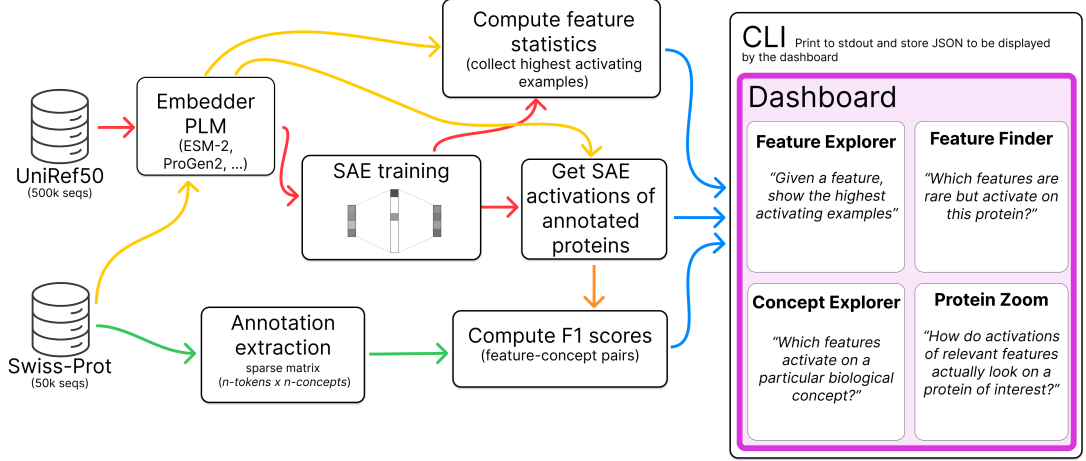


Figure 3.1 Schema of the interpretability pipeline, including feature analysis modes through command-line interface (CLI) and interactive dashboard. SAE diagram retrieved on 15.7.2026 from https://adamkarvonen.github.io/machine_learning/2024/06/11/sae-intuitions.html. PLM embedder is used for different purposes: first for generating training data for SAEs, and second for embedding annotated sequences used for analysis, as marked by the red and yellow-colored arrows. The remaining arrow colors have no meaning.

sequence length. We include the terminus tokens because prior testing showed that sequences in the format `1<seq>2` achieve lower negative log-likelihood than bare sequences. Note that ProGen2 can also encode reversed sequences in the format `2<rev-seq>1`, but we chose not to use this mode for simplicity and compatibility with the rest of the pipeline.

We randomly selected 500,000 sequences from UniRef50, which in total contains roughly 39 million proteins, as of release 2026_02. We split them into 500 shards of 1,000 sequences each, reserving 50 shards as the test set. The training split contained roughly 110M tokens in total, corresponding to an average of approximately 245 tokens per sequence. We extracted embeddings using ProGen2-large (2.7B parameters) and ProGen2-small (151M parameters). ProGen2-large has a hidden dimension of 2,560 and ProGen2-small has 1,024. Embeddings were stored in the default 16-bit precision. For ProGen2-large, we extracted embeddings from layer 9 and layer 24, in order to have one layer from the first half and one layer from the second half of the 32-layer model. For ProGen2-small, we chose layer 7 out of 12, following the InterPLM finding that layer in ESM-2 at the same depth gave the best concept coverage [9]. The total size of the stored embeddings was 526 GB for ProGen2-large (both layers combined) and 211 GB for ProGen2-small.

We also improved the embedding script to support multiple GPUs. Each GPU acts as a worker, and a multiprocessing executor assigns shards to workers in parallel. If only one GPU or no GPU is available, the script falls back to the original sequential implementation. In addition, the batch size is automatically halved when a process runs out of GPU memory, and shards that have already been processed are skipped automatically so interrupted runs can be resumed.

Training SAEs

The next step is to train sparse autoencoders on the extracted embeddings. We use SAE implementations from the `dictionary_learning` package [52], which provides all common SAE architectures and activation functions. The InterPLM pipeline implements trainers for these architectures, along with a dataloader for the sharded data, evaluation and fidelity computations, and training monitoring through Weights & Biases (W&B).

The most important hyperparameters shared across SAE types are the expansion factor (which determines how many times larger the SAE hidden dimension is compared to the input embedding dimension), the learning rate, the batch size, the number of training steps, and the learning rate decay schedule. We experimented with the classical ReLU SAE, which was also used in the original InterPLM work, and later with batch top-k, which proved easier to train.

For the ReLU SAE, the L1 penalty is the most critical hyperparameter, as it balances the reconstruction loss against the sparsity loss. The L1 penalty is warmed up from zero at the start of training and decayed towards the end. A known issue with L1-based sparsity is that it penalizes the magnitude of activations, which in practice causes the model to learn the correct directions in activation space but with much smaller magnitudes than expected. The original InterPLM trainer supports dead neuron resampling as an optional mechanism to partially address this. We enabled it by default in our training configuration, which introduces an additional hyperparameter to tune.

Batch top-k [44] differs in its activation function. On average across the batch, it enforces that k neurons are active per token. With k directly available as a hyperparameter, sparsity is much more convenient to control than with an L1 penalty. A ReLU is applied before the top-k selection, so all activated neuron values are non-negative. To address the dead neuron problem, batch top-k uses an auxiliary loss term weighted by `aux_k_alpha`, which identifies dead neurons and encourages them to contribute to reconstructions. At inference time, when the batch size is 1, the batch top-k selection cannot be applied directly. Instead, a learned threshold is used. During training, the threshold is maintained as an exponential moving average of the minimum activation among the k selected neurons per batch.

On top of the existing InterPLM implementation, we added an option to run the fidelity evaluation every n training steps on a subset of the data, rather than only once at the end of training on the full test set. Fidelity is defined as the percentage of loss recovered,

$$\text{fidelity} = 1 - \frac{\mathcal{L}_{\text{recon}} - \mathcal{L}_{\text{orig}}}{\mathcal{L}_{\text{zero}} - \mathcal{L}_{\text{orig}}},$$

where $\mathcal{L}_{\text{orig}}$ is the loss of the base model on the data, $\mathcal{L}_{\text{recon}}$ is the model’s loss when we replace the residual stream with SAE reconstructions, and $\mathcal{L}_{\text{zero}}$ is the zero-ablated loss, meaning we replace the residual stream with all zeroes. Computing fidelity requires running the PLM twice, once unmodified and once with the SAE reconstruction, so running it mid-training on a subset saves considerable time while still giving us a useful signal. We also cache the $\mathcal{L}_{\text{orig}}$ and $\mathcal{L}_{\text{zero}}$ values, since both are a function only of the model, the layer, and the data. We hash all three of these together to uniquely key the cached values.

The training metrics logged to W&B include: the fidelity metric, the reconstruction loss (cross-entropy), MSE, the total loss and its individual components, and the fraction of dead neurons and of highly active neurons. These metrics are ultimately proxies for what we care about, which is interpretability and *steerability*, as we discuss in the following chapter. We will also present the training results and empirical observations in the following chapter.

Normalizing SAE activations

In order to make feature activation values comparable across features, we normalize SAE activations so that values are roughly in the range $[0, 1]$ for each feature. We do this by passing a random subset of UniRef50 sequences through the SAE and recording the maximum activation value per feature. This maximum is stored as a vector in the model buffer and used to divide activations at inference time. Note that if a feature’s activation exceeds the observed maximum, the resulting normalized value will be greater than 1.

Extracting annotations

The InterPLM pipeline extracts protein annotations from Swiss-Prot [29] entries and stores them as a sparse matrix of shape $\text{num-tokens} \times \text{num-concepts}$, where num-concepts is the number of unique annotation types we consider. Concepts include domain types (e.g., `domain_core_binding`, `domain_sushi`), active and binding sites, and positional features such as membership in a secondary structure element. In Swiss-Prot, domain annotations are specified as residue ranges, for example: "DOMAIN 442..557; /note="STAS"; /evidence="EC0:..." marks a STAS domain spanning residues 442 to 557.

The annotation extraction script takes a TSV file of Swiss-Prot annotations as input (compressed or uncompressed). It shards the data and saves, for each shard: `protein_data.tsv` with the raw Swiss-Prot data; `aa_metadata.csv` with the UniProt ID, amino acid, and position within the sequence; and `aa_concepts.npz`, a compressed sparse matrix of shape $\text{num-tokens} \times \text{num-concepts}$. Categorical concepts are included only if they appear at least a minimum number of times, which we set to 10 via the `-min-required-instances` parameter. We also exclude sequences longer than 1022 tokens, which is the context window of both ProGen2 and ESM-2, accounting for the 1 and 2 terminus tokens.

The original script assumed that all annotation types use the keyword `note` to describe the annotation. This is true for many categories, however, binding site annotations use the `ligand` keyword instead, e.g.: "BINDING 83; /ligand="ATP"; /ligand_id="ChEBI:CHEBI:30616"; /evidence="...". As a result, the original implementation missed all binding site concepts, despite the original paper claiming to have searched for them. We fixed this parsing bug and after the fix, we found ESM-2-8M layer 4 (which is the flagship model layer) contains 5 features which identify Heme-C binding site with F1-per-domain greater than 0.5. We define the notion of domain-adjusted F1 score, originally used by InterPLM in section 3.2.1. Furthermore, We found that layer 9 from ESM2-650M contains features which identify concepts for heme A3 and cyclic adenosine monophosphate binding sites and layer 24 of the same model also identifies heme C binding sites. We did not investigate other layers.

Another issue was that sometimes an index is appended to a domain name to distinguish multiple occurrences within the same protein, for example "CBS 1" and "CBS 2" for two instances of the CBS domain. The original implementation treated these as two distinct concept types. While this could in principle be intentional (to identify features that fire specifically on the second occurrence of a domain), it is more likely an oversight. We observe that the same SAE feature typically fires on all occurrences of the same domain, so we updated the code to strip trailing numerical indices from domain names and merge such annotations. Note that Swiss-Prot is not fully consistent in this respect. Sometimes indices are included even for single-occurrence domains, and sometimes they are omitted even when multiple occurrences are present.

We also parallelized the shard processing in the extraction script using a process pool executor, so multiple shards are processed simultaneously rather than sequentially as in the original, similarly to the improved implementation of embedding extraction in the first step of the pipeline.

The next step is to split the annotated shards into a validation split and a test split. The validation split is used to select a binarization threshold per feature, i.e., the activation value above which a feature is considered active on a given residue. The test split is then used to report final concept association metrics using the thresholds determined on validation data. This step also applies a second round of filtering, retaining only concepts that either appear across at least a minimum number of domain instances or cover at least a minimum number of annotated residues. We use a minimum of 25 domain instances per concept (the default) but only 10 amino acids per concept, compared to the default of 1000 from IntePLM, because we are specifically interested in rare annotations. The two conditions are combined with a logical OR, not AND.

Since we were specifically interested in binding site concepts, we bypassed the minimum occurrence filter for those annotations. Together with the fixed parsing, this allowed us to include 291 concepts in the analysis that the original work did not consider. Note also that we used a different random subset of Swiss-Prot, which may explain some additional differences.

Embedding annotations

The next step is to compute PLM embeddings for the annotated Swiss-Prot sequences. These embeddings are cached in order to compute their sparse SAE representations efficiently, without having to run the PLM again. We extract embeddings from the same layer as for the SAE training data.

We improved the embedding script by checking which shards have already been embedded and skipping them, and by supporting parallel embedding across multiple GPUs, using the same multiprocessing approach as for the training data. The batch size is also automatically halved on an out-of-memory error.

As in the original publication, we randomly selected 50,000 proteins, sharded into files of 1000 sequences each.

Computing concept-feature pair scores

At this point we have PLM embeddings and sparse concept annotation matrices for the annotated proteins. To quantify the association between concepts and

Algorithm 1 Original concept-feature scoring (feature-chunk loop)

Require: embeddings $A \in \mathbb{R}^{T \times d}$, labels $L \in \mathbb{Z}_{\geq 0}^{T \times C}$, SAE, thresholds $\tau_1, \dots, \tau_K$

- 1: initialise TP, FP, $\text{TP}^{\text{dom}} \leftarrow \mathbf{0}_{C \times F \times K}$
- 2: **for** each feature chunk $\mathcal{F} \subseteq \{1, \dots, F\}$ of size ≈ 250 **do** $\triangleright \sim 40$ chunks;
GPU $\rightarrow$ CPU each
- 3: $\hat{A} \leftarrow \text{SAE.encode}(A)[:, \mathcal{F}]; \quad \hat{A}_{\text{sp}} \leftarrow \text{to_sparse}(\hat{A}.\text{cpu}())$
- 4: **for** $k = 1$ **to** K **do**
- 5: $B \leftarrow \mathbf{1}[\hat{A}_{\text{sp}} > \tau_k]$ $\triangleright$ binarize, CPU sparse
- 6: **for** $c = 1$ **to** C **do**
- 7: $\text{TP}[c, \mathcal{F}, k] \leftarrow (B \odot (L_{:,c} > 0)).\text{sum}(0)$
- 8: $\text{FP}[c, \mathcal{F}, k] \leftarrow (B \odot (L_{:,c} = 0)).\text{sum}(0)$
- 9: **if** not aa_level[c] **then**
- 10: $\text{TP}^{\text{dom}}[c, \mathcal{F}, k] \leftarrow |\text{unique_nonzero}(B \odot L_{:,c})|$

features, we compute F1 scores for all pairs. We treat the concept annotations as the ground truth labels and the sparse autoencoder activated features as our predictions. Since F1 score requires counting true positives, false positives, and false negatives, we first need to binarize the features. If it fires above a certain threshold, we consider it active on that residue, otherwise inactive. We compute F1 scores for all feature-concept pairs, which involves on the order of at least 10,000 features and roughly 740 concepts.

The computation loops over candidate binarization thresholds and, for each, computes matrices of true positives and false positives with shape num-features $\times$ num-concepts, which allows us to compute precision, recall, and F1 for every pair. One subtlety is that many SAE features fire on a particular sub-region of a domain rather than the domain as a whole (for example, on the third and sixth residues within the first helix of a domain). Such a feature would have perfect precision but very poor residue-level recall, despite being clearly interpretable. The InterPLM pipeline therefore uses a domain-level recall instead: recall-per-domain = TP/num-domains, where TP is the number of domain instances in which the feature fires at least once. The F1-per-domain score is the harmonic mean of precision and recall-per-domain. We consider a concept to be identified if at least one feature achieves F1-per-domain greater than 0.5.

Formally, the computation reduces to computing three tensors of shape (C, F, K) , where C is the number of concepts, F the number of SAE features, and K the number of candidate thresholds. Given feature activations $A \in \mathbb{R}^{T \times F}$ and a label matrix $L \in \mathbb{Z}_{\geq 0}^{T \times C}$ (where $L_{t,c} = 0$ means concept c is absent at token t , and $L_{t,c} = d$ encodes domain instance d), we need:

$$\begin{aligned} \text{TP}_{c,f,k} &= \sum_t \mathbf{1}[A_{t,f} > \tau_k] \cdot \mathbf{1}[L_{t,c} > 0], \\ \text{FP}_{c,f,k} &= \sum_t \mathbf{1}[A_{t,f} > \tau_k] \cdot \mathbf{1}[L_{t,c} = 0], \\ \text{TP}_{c,f,k}^{\text{dom}} &= \left| \{d > 0 : \exists t, L_{t,c} = d \wedge A_{t,f} > \tau_k\} \right|. \end{aligned}$$

Algorithm 1 shows the structure of the original implementation. Algorithm 2 shows the rewritten version.

Algorithm 2 Optimised concept-feature scoring (token-chunk loop)

Require: embeddings A , binary labels $\tilde{L} = \mathbf{1}[L > 0]$, domain indicators $M^{(c)}$ for non-AA concepts, SAE, thresholds $\tau_1, \dots, \tau_K$

- 1: initialise $\text{TP} \leftarrow \mathbf{0}_{C \times F \times K}$, $\text{total_active} \leftarrow \mathbf{0}_{F \times K}$ (GPU)
- 2: initialise $E_c \leftarrow \mathbf{0}_{K \times |\mathcal{D}_c| \times F}$ (bool) for each non-AA concept c
- 3: **for** $b = 0$ **to** $\lceil T/B \rceil - 1$ **do** $\triangleright B = 4096$ tokens; all F features stay on GPU
- 4: $\hat{A} \leftarrow \text{SAE.encode}(A[bB:(b+1)B], \text{normalise}=\text{True})$ $\triangleright (B, F)$
- 5: **for** $k = 1$ **to** K **do**
- 6: $B_k \leftarrow \mathbf{1}[\hat{A} > \tau_k]$ $\triangleright (B, F)$
- 7: $\text{TP}[:, :, k] += \tilde{L}[bB:(b+1)B]^\top \cdot B_k$ $\triangleright (C, B) \times (B, F) \rightarrow (C, F)$
- 8: $\text{total_active}[:, k] += B_k.\text{sum}(0)$
- 9: **for** each non-AA concept c **do**
- 10: $E_c[k] |= (M^{(c)}[bB:(b+1)B]^\top \cdot B_k > 0)$ $\triangleright$ boolean OR
- 11: $\text{FP} \leftarrow \text{total_active} - \text{TP}$
- 12: $\text{TP}^{\text{dom}}[c] \leftarrow E_c.\text{sum}(\text{dim}=1)$ for each non-AA concept c

The original implementation had three nested loops over feature chunks, thresholds, and concepts surrounding sparse CPU matrix operations. The inner loop executed $40 \cdot K \cdot C = 40 \cdot 5 \cdot 740 = 148,000$ sparse element-wise multiplies and reductions on CPU, each allocating a new sparse matrix and paying full Python/SciPy overhead. Domain counting was even slower, since it called a Python `set()` over each feature column’s data, executed approximately 10^7 times per shard. For a SAE with 10,240 features, 740 concepts, and 5 threshold values, this took approximately 2.5 hours per shard across the 25 validation and 25 test shards.

We rewrote the computation in three steps. First, we eliminated the inner concept loop by observing that the TP and FP matrices for all concepts at once reduce to a single matrix product. Let $B_k \in \{0, 1\}^{T \times F}$ be the binarized activations at threshold τ_k and $\tilde{L} \in \{0, 1\}^{T \times C}$ be the binary label matrix $\mathbf{1}[L > 0]$. Then:

$$\text{TP}_{[:, :, k]} = \tilde{L}^\top B_k \in \mathbb{R}^{C \times F}, \quad \text{FP}_{[:, :, k]} = \mathbf{1}^\top B_k - \text{TP}_{[:, :, k]}.$$

This collapses $C = 740$ inner iterations into one matrix product per threshold. Second, we inverted the loop order: instead of processing all T tokens for 40 feature chunks, we process all F features for $\lceil T/B \rceil$ token chunks (with $B = 4,096$), computing the accumulation on GPU as a dense matmul $\tilde{L}_{[b:b+B]}^\top \cdot B_k^{[b:b+B]}$ of shape $(C, B) \times (B, F)$. This eliminates the 40 repeated GPU→CPU tensor copies and sparse matrix constructions of the original. Third, for TP^{dom} , we replaced the Python `set()` loop with a thresholded matmul and a persistent boolean OR accumulation: for each non-AA-level concept c with domain indicator $M^{(c)} \in \{0, 1\}^{T \times |\mathcal{D}_c|}$, we maintain a tensor $E_{c,k} \in \{0, 1\}^{|\mathcal{D}_c| \times F}$ and update $E_{c,k} |= (M_{[b:b+B]}^{(c)\top} B_k^{[b:b+B]} > 0)$ per chunk. The final $\text{TP}_{c,f,k}^{\text{dom}} = \sum_d E_{c,k}[d, f]$.

We also found that features must be normalized before binarization. Without normalization, the BatchTopK SAE’s learned scalar threshold already zeros out most activations, causing all candidate threshold values to collapse to the same result. Normalizing by the per-feature maximum (from the normalization step) ensures the thresholds act as meaningful fractions of each feature’s peak activation.

The rewritten implementation produces identical results to the original. The time per shard dropped from 2.5 hours to approximately 50 seconds, a **180x** speedup.

Collecting feature activations

The final step of the pipeline is to collect, for every feature, the examples that later populate the dashboard described in the next section. For each feature we cache the ten proteins that activate it most strongly, together with a handful of examples sampled from three lower activation quantiles (0–40%, 40–80%, and 80–100% of the feature’s observed maximum), so that the dashboard can also show weaker, borderline activations and not only the cherry-picked top ones. We also record two summary statistics per feature: the percentage of proteins in which the feature is active at all (using a default threshold of 0.05 to count a residue as "activated"), and the average percentage of residues activated within a protein once the feature is present there. All of these results, together with the per-feature maximum activations already used for normalization in the previous step, are saved to disk and loaded directly by the dashboard.

3.2 Interpreting the Features

We have now computed F1 and F1-per-domain for every concept-feature pair, and by default we mark a concept as identified if at least one feature reaches an F1-per-domain above 0.5. This quantitative view gives us a good overview of how well the SAE performs, but it misses many features that are not aligned with any Swiss-Prot annotation but may be still interpretable.

The original InterPLM dashboards came with a default *feature explorer* view, which lets the user select a feature and shows its top activating examples. This does not help us browse for features of interest which are firing on a certain type of proteins. We, or our users, might instead want to see which features fire on a specific concept, for example a particular binding site or domain we care about. Or we might have a query protein of interest and want to know which features firing in this (possibly unannotated) protein are highly specific to it, which could in turn help us find similar proteins, possibly annotated ones, and so suggest missing annotations for our protein.

To address this, we implemented two new ways of exploring the data through interpreted SAE features. The first is the so-called *per-concept* view, where we start from a query concept and find the top features and top examples related to it. The second is the *per-protein* view, where for a query protein we identify the most specific features firing in it and provide possible interpretations for them. Each of these two query types has a “computational core”, a Python script that stores the results of the query in a structured JSON format. Queries can be submitted either through the CLI or directly from a form in the dashboard, and results are cached. After a CLI submission, once the job finishes, or once the result is loaded from cache, the output is printed to stdout, or more conveniently, results can be viewed and browsed interactively in the dashboard.

3.2.1 Per-concept Analysis

The per-concept analysis takes a concept of interest and returns the SAE features associated with it. It answers the question: “for a given concept, which features are related to it, and when we rank the residues of a protein by the magnitude of activation of that feature, how highly does it rank the residues carrying the desired concept annotation?” This is useful for investigating which features the model uses to represent a particular biological annotation, and especially useful for rare and sparse concepts.

There are two modes of this analysis, which we call *F1-guided* and *annotation enrichment*. The former is the simpler approach, and it uses the CSV of precomputed F1 scores for all feature-concept-threshold triplets from the pipeline. We simply query it to find which feature, and at which threshold, achieves the highest F1 (or F1-per-domain) for our concept, and then select that feature together with its max-activating examples.

However, for very rare concepts, even the domain-adjusted F1 may not surpass the 0.5 threshold. Another issue is that missing annotations can pull F1 down even when the feature is actually well calibrated, and that we first have to find a threshold to mark the features as active or not. The *annotation enrichment* mode tries to “bootstrap” around the missing annotations by running the SAE forward across the annotated test set and collecting two vectors of shape `num_features: concept_sum`, the sum of activations of all features on residues annotated with the query concept, and `background_sum`, the sum of activations of all features on residues that are not annotated. We then average these sums to obtain per-residue means and compute an enrichment ratio for every feature as `enrichment_ratio = mean_at_concept / mean_at_background`. The advantage over the F1-guided mode is that we no longer need to pick a threshold at which to binarize the feature, and can use the raw activation values instead. We then retrieve the top n features by enrichment ratio, with a default of $n = 5$, same as in the F1-guided mode.

This mode also avoids a subtler problem with the F1-guided one. InterPLM authors switched to recall-per-domain, and therefore F1-per-domain, to deal with features that otherwise looked like they had too many false negatives, but kept the definition of precision unchanged, $\text{precision} = TP / (TP + FP)$, reasoning that since features typically fire on fewer residues than there are concept annotations, false positives should not be too large a problem. But we may have missed cases where a feature has low precision and good recall and dismissed it. Consider a feature f that fires at value 1.0 on 50 annotated residues, fires at an average of 0.5 on 5000 unannotated residues, and does not fire (0.0) on the remaining 500,000 residues. At a threshold of, say, 0.4, its precision would be $50 / (50 + 5000) \approx 0.01 = 1\%$, which would ruin its F1 score even at a recall of 1, giving $F_1 \approx 0.02$ (and this is, again, why we sweep over the thresholds in the first place). Under the enrichment mode, however, we instead get $\text{concept} = (1.0 \times 50) / 50 = 1$, $\text{background} = (5000 \times 0.5) / 505,000 \approx 0.005$, and $\text{enrichment} = \text{concept} / \text{background} = 1 / 0.005 = 200$.

On a technical note, we also implement caching for this analysis. We precompute the sum of all SAE activations for each concept (a 2D matrix of size `num_concepts × dict_size`), the concept counts, and the total sum of activations (a 1D vector of size `dict_size`), which is used for the background frequency. The cache is created lazily on the first query, so all subsequent queries can be answered quickly.

Regardless of which mode we use to find the top features for a concept, both produce a ranked list of top features for the query concept as output. We evaluate this list with several retrieval-inspired metrics. If we think of the raw activation values of a certain feature across the residues of a protein as a retrieval scoring function, how highly do labeled residues score? For each protein we report: hits@k, prec@k, rec@k, MRR (mean reciprocal rank), and AUPRC, computed over the highest activating examples. We also report aggregate metrics, essentially the average of these per-protein metrics across all proteins labeled with the concept, together with an AUPRC baseline and a hits@k baseline, which are the values we would expect if we ranked the residues of labeled proteins at random.

3.2.2 Per-protein Analysis

The per-protein analysis is conceptually simpler. It serves as a simple entry point for further feature exploration, and is especially useful when we have an unannotated protein of interest. It aims to answer the question: “What is distinctive or notable about this specific protein?” by finding features that fire often in this protein but not often across the whole dataset.

The per-protein script takes a protein identifier (a UniProt ID) or a raw amino acid sequence, computes a forward pass through the PLM and the SAE, and returns the features that are active in that protein. For each active feature, it reports two characterizations. Here, we consider a feature as active if it fires above the threshold used for computing F1 scores with annotated concepts. The first is cross-dataset specificity: a feature is classified as *specific* if it fires in fewer than 10% of proteins in the dataset, *semi-specific* if it fires in between 10% and 50%, and *general* above 50%. The second is the within-protein activation pattern: a feature is *focal* if it activates on fewer than 20% of residues in the protein, *broad* above 50%, and *semi-focal* in between. For each feature we also find the concept with the highest F1 score in the analysis table and label the feature with that concept, also reporting the top F1 itself so we can assess how reliable the label is. By default, results are printed to stdout and also saved as a JSON file for downstream use in the dashboard.

3.2.3 Dashboard

The results can be explored interactively in a dashboard implemented in Streamlit [53], a Python library for building simple web applications. The F1 scores described above are one way to interpret a model’s hidden states, but many interesting features may not map cleanly onto Swiss-Prot concepts, either because the annotation does not exist, or because it is incomplete or inaccurate. The original authors addressed this by building a dashboard for manual feature browsing.

The original dashboard had a single page, called the *Feature Explorer*. It shows a scatter plot in which each feature is a point, with the x axis marking the percentage of proteins in which that feature is active and the y axis marking the percentage of active residues within those proteins. Selecting a feature shows the top activating proteins, the activation profile as a function of sequence position, and a three-dimensional structure from the AlphaFold Database in which residues

are colored by activation intensity. This page is shown in figure 3.2, part A, with the visualization for feature **f/2** and its highest activating example.

We added several new pages to the dashboard, so that users can interactively browse the results and launch runs of the new analysis modes we implemented and described in the previous section.

Concept explorer

The second view we added, called the *Concept Explorer*, is the visualizer for the per-concept analysis described above, and it lets the user browse concepts rather than features. It helps answer which features are related to a concept of interest, and how well those features localize to the residues that carry that concept’s annotation. In the left sidebar, the user submits a new query by specifying a concept name as a string, which can also be a substring of a concept name, in which case we search by substring match. If there is more than one match, we return an error listing the possible matches and ask the user to refine the query. The other parameters are the annotations directory, the analysis mode (annotation-enrichment or F1-guided), how many features to return, and how many examples per feature to show. After submission, we first search the cache and try to load previous results, and if the query is not cached, we run the analysis, which itself loads from another cache holding the precomputed enrichment scores. The sidebar also lists previous runs of the per-concept analysis, so their results can be loaded instantly.

The top of the page shows a table with the top features and their statistics, and below it, for each feature, the top examples. The examples are ranked either by activation magnitude or by best “localization”, which we define as the within-protein AUPRC, where the positive labels are the annotated residues and the scores are the raw activation values. Ranking by magnitude surfaces the proteins in which the feature fires most strongly, whereas ranking by localization surfaces the proteins in which the feature fires specifically on the annotated region rather than merely somewhere in the sequence. The “best localization” mode only returns proteins with at least one annotated residue. By default we show the activation profile only as a static PNG image generated with matplotlib, and we do not show the structure. The reason is that rendering many interactive Streamlit plots and 3D structures at once quickly makes the page laggy and unresponsive, so the interactive activation visualization across sequence positions and the 3D structures are only shown (or hidden) once the user presses a button. At the top of this page, and next to each shown structure, there is a link to the *Protein Zoom* page. The concept explorer is shown in figure 3.2, part B.

Protein Zoom

The *Protein Zoom* page is an extension of the concept explorer, used for “zooming in” on where certain features fire within a particular protein and how that compares to its annotated regions. It has submission forms in the left sidebar, where the required parameters are the protein’s UniProt ID and a comma-separated list of feature IDs, and the concept name can optionally be specified as well. When we click the *Protein Zoom* button next to a protein example on the concept explorer page, these forms are automatically pre-filled with that protein’s

ID, the top features found by the concept explorer, and the annotation for the concept we searched for.

After the analysis finishes, we can see the activation profiles of the requested features for this protein all at once, as shown in figure 3.2, part C. The green regions mark the annotated regions, and the bar graphs are the activation values across the sequence. Optionally, we can view the activations overlaid on one another, rather than in the stacked view shown here.

Feature Finder

The last page, the *Feature Finder*, visualizes the results of the per-protein analysis. It helps answer what is distinctive about a particular protein, by showing the features that fire least frequently across the dataset from which we collected the samples. It labels a feature as "specific" if it is active in fewer than 10% of the proteins, and it also assigns an activation-pattern label of focal, semi-focal, or general, based on the percentage of residues in the query protein on which the feature is active. This view is meant as a guide for selecting potentially interesting features to use as queries on the other pages of the dashboard.

The second table on the page has rows for all concepts, including amino acids, present in the protein. The table has columns for the number of annotated residues, the number of active features, the number of overlapping positions, and the intersection over union fraction, and the top feature by F1 for that concept.

Overall, this page has the simplest UI, with a form in the left sidebar under the main menu, and only static tables in the main panel. It can be seen in figure 3.2, part D.

3.3 Activation Steering

One of the central questions in this thesis is, once we identify SAE features that track a given property, whether those features merely describe the property or actually cause it.

Our steering pipeline takes a base model (here ProGen2), an SAE trained on one of its layers, a prefix to continue from, and a specification of which feature to steer and how. To intervene on the model's computation we register a forward hook on the transformer block corresponding to the SAE's training layer. A forward hook is a function PyTorch calls right after a module finishes its forward pass, with the module's output as an argument, and if the hook returns a value, that value replaces the output for every layer downstream, which is how we substitute in the steered residual stream. Locating the right block takes some care: `hidden_states[k]` is defined as the output of block `h[k-1]`, not `h[k]`, so to hook the same activations the SAE was trained on we must hook `h[layer_idx - 1]`. Because of this off-by-one, we check before every steering run that the hook is reading the right layer. Once the hidden state is captured, we encode it with the SAE, modify the feature activations of interest, and decode the result back into hidden-state space before returning it to the model. A specification such as `1234:5.0` clamps feature 1234 to five times its highest observed activation. The default unit is this multiple of the maximum, but a raw value independent of the maximum can be given instead. The multiple can also be negative. Activations

are always non-negative, so a negative multiple simply subtracts a multiple of the corresponding row of the SAE decoder matrix from the residual stream. Besides clamping, the script can also ablate a feature (set it to zero) or add a value on top of its current activation. Tokens are then drawn by multinomial sampling, weighted by the model’s output probabilities.

The intervention can be applied in one of two ways. The default is the `with_error` mode which is described in the ML background chapter, but we also implemented a `direct` mode which just replaces the residual stream with steered SAE reconstruction, without adding back the error term.

Sequences are generated autoregressively with a simple left-to-right sampling loop, since ProGen2’s remote model configuration is not compatible with Hugging Face’s `generate()`. To keep this affordable we reuse the model’s attention key-value cache across steps: the prompt is processed once, and each subsequent step only feeds the newest token, with the hook keeping track of the token’s absolute position so that it keeps steering the newly generated positions correctly. This is numerically equivalent to re-feeding the whole growing sequence at every step, since the hook only modifies a layer’s output, which can only affect deeper layers within the same forward pass and not earlier ones. We also restrict steering to the part of the sequence after the prompt, so that when a prefix is given, its own hidden states pass through the hook unmodified and only the generated continuation is steered.

Previous work [10] recommends clamp multiples between -10 and 10 , and we found the same range usable, but highly depending on the value of k in the SAE’s activation function. Steering a feature up, to increase its associated property in the sequence, worked more reliably than anti-steering it down to suppress the property by clamping to a negative value. Clamps of large magnitude tend to produce degenerate outputs, for example an end-of-sequence token appearing too early, a high NLL loss, or long stretches of a single amino acid. We go into more detail on the choice of clamp values with concrete experiments in the next chapter.

4 Results

In this chapter we present the results of training SAEs, comparing the trained ProGen2 [4] SAEs against the pretrained ones for ESM-2 [2] from InterPLM [9], demonstrate the practical usage of our dashboard and analysis modes by finding interesting features in the sparse space, and finally attempt activation steering for several properties.

4.1 Training Sparse Autoencoders

In this section we describe how training the sparse autoencoders (SAEs) went in practice and discuss our empirical observations about which hyperparameters turned out to matter the most. We ran a large number of experiments, since a single SAE was relatively cheap to train, and there were several knobs to tune. The first choice was the SAE architecture itself, or more precisely the activation function that enforces sparsity. We experimented with two variants: a plain ReLU activation regularized with an L1 penalty, and BatchTopK.

The ReLU variant was tricky to train. Our early runs failed due to one of two reasons. Either the SAE had too many dead features, meaning features that never fire on any input, or its sparsity was too high. For a few initial runs, we did not realize that resampling of dead neurons was turned off by default. Resampling periodically reinitializes the weights of features that have stopped firing, so that gradient can flow through them again and they get another chance to learn something useful. Once we enabled it, the dead-feature problem improved a lot. However, the main issue of ReLU-activated SAEs is that sparsity is controlled only indirectly, through the strength of the L1 penalty (λ_1). The penalty influences how sparse the features end up, and the resulting sparsity is measured only after training as the L0 norm, which is the mean number of active features per token. The original InterPLM work [9] reported this L0 as its sparsity measure. We can avoid this indirection completely by using a top-k activation, where we set the number of active features k directly.

For this reason, and because it had performed better in earlier work, most of our models use BatchTopK instead of ReLU. We did not try the plain top-k variant. BatchTopK enforces the constraint across the whole batch rather than per token, so each token gets k active features on average while individual tokens may have more or fewer active features. We still trained and report a few ReLU SAEs for comparison, but BatchTopK is the main architecture used throughout for later experiments in this work. We trained SAEs only for ProGen2-large layers 9 and 24 and for ProGen2-small layer 7, and we focused mostly on the larger model, where we expected better performance and more identified features. A single run of training SAE for the larger model took about three hours, and about 90 minutes for the smaller one on one H100 GPU. We typically defined a grid of hyperparameters and submitted the job overnight. The main bottleneck was the fidelity computation. In the original implementation it ran too often and on too much data, so we added parameters controlling how frequently it is evaluated (every 20k steps by default) and on how many tokens (200 batches of 1024 tokens was our default), which sped up training (specifically evaluation)

considerably. For ProGen2-large we used an expansion factor of 4, in order to match the dictionary size of 10,240 that the ESM-2 SAEs from InterPLM use, which we compare against. The two ProGen2-small SAEs instead use expansion factors of 16 and 4, giving dictionaries of 16,384 and 4,096. Table 4.1 lists the hyperparameters of several of the trained models. All of them are optimized with AdamW and gradient clipping at 1.0, and share a learning-rate schedule that warms up linearly to the peak rate over the first 10% of training steps and then decays to zero starting from the listed decay-start step, which is another hyperparameter to tune. The sparsity column reports either k , the number of active features per token for BatchTopK, or the L1 penalty λ_1 for ReLU. We experimented with all of these settings: batch size, learning rate, k or the L1 penalty depending on the architecture, and the mechanism for mitigating dead features. BatchTopK adds an auxiliary reconstruction loss on the dead features with a coefficient α , while ReLU uses periodic resampling of dead neurons after a certain number of steps of those neurons being dead.

The main difficulty was not any individual hyperparameter but the objective we were implicitly optimizing for. Through most of the sweeps we tuned for high fidelity, i.e. the fraction of the language-model cross-entropy loss that is recovered when we patch the SAE reconstruction back into the model. Fidelity is convenient because it is cheap to compute during training, but it turned out to correlate poorly with what we actually care about, the interpretability of the features, which is itself somewhat subjective to judge and depends on the presence and quality of annotations. We only noticed this after training many SAEs and running the concept analysis, where we found that the sparser SAEs identified more concepts even when their fidelity was down to 0%. Note, that this runs against what optimizing for fidelity encourages, since denser reconstructions (higher k or L_0 metric) tend to recover more of the loss. Overall we found the training hyperparameters tricky to tune, and the downstream interpretability has no good proxy among the training metrics, so the SAEs are best judged individually. We report and discuss the computed metrics in the following section.

Model	Layer	k / λ_1	LR	Batch	Dead-feat. mitig.
ESM2-8M	4	$\lambda_1=0.1$	10^{-6}	2048	-
ESM2-650M	9	$\lambda_1=0.06$	10^{-5}	2048	-
ESM2-650M	24	$\lambda_1=0.05$	5×10^{-5}	2048	-
ProGen2-S	7	$k=50$	10^{-4}	512	aux- k $\alpha=0.031$
ProGen2-S	7	$k=96$	2×10^{-4}	512	aux- k $\alpha=0.031$
ProGen2-L	9	$\lambda_1=0.007$	5×10^{-6}	512	resample @ 20k
ProGen2-L	9	$\lambda_1=0.04$	5×10^{-6}	512	resample @ 20k
ProGen2-L	9	$k=64$	5×10^{-5}	512	aux- k $\alpha=0.031$
ProGen2-L	24	$k=10$	5×10^{-5}	1024	aux- k $\alpha=0.016$
ProGen2-L	24	$k=64$	5×10^{-5}	1024	aux- k $\alpha=0.031$
ProGen2-L	24	$k=350$	5×10^{-5}	512	aux- k $\alpha=0.031$

Table 4.1 Training hyperparameters for the ProGen2 SAEs we trained (the ESM-2 SAEs we compare against are pretrained from InterPLM).

4.2 Generative vs. Masked Language Models

The two model families we compare were trained with different objectives. ESM-2 is a masked language model, with a bidirectional encoder transformer architecture that predicts held-out residues from the full surrounding context on both sides. ProGen2 is a causal, autoregressive (decoder transformer) model that predicts each residue only from the ones before it. We compare the two families on how many of the annotated concepts their features detect and on how clean those features are, and afterwards we break the comparison down by concept type.

Out of curiosity, we also compare the language modeling loss of the two model families. The ProGen2 paper reports cross-entropy losses for its models, but the ESM-2 paper does not, so we measure both ourselves on a held-out set of UniRef50 sequences (the same set we use for fidelity during SAE training). Each model is scored on its own training objective: next-token prediction for the decoder ProGen2, and masked-token prediction with 15% of residues masked for the encoder ESM-2. The results turn out to be quite similar across the two families, as shown in the table below. Interestingly, even the smallest ESM-2 model (8M) reaches a lower loss than the large ProGen2 (2.7B), most likely because predicting a masked residue from both sides of the context is an easier task than predicting the next one from the left alone. Alongside the cross-entropy loss (in nats) we report perplexity, defined as the exponential of the cross-entropy, which has a more intuitive interpretation, measuring how many tokens the model is choosing between, on average, for each prediction. A perfect model would have loss 0 and perplexity 1, so it would only ever consider a single token. With only 20 amino acids to pick from, a perplexity roughly 7 to 14 may seem quite high, but it is what both model families land on, and they still perform well on their respective tasks.

Model	Cross-entropy	Perplexity
ESM-2 (8M)	2.39	10.89
ESM-2 (650M)	1.97	7.19
ProGen2-small	2.62	13.72
ProGen2-large	2.50	12.13

Table 4.2 collects, for several trained SAEs, their training sparsity, reconstruction fidelity, and three feature-level interpretability metrics. *Fidelity* is the fraction of the language-model loss recovered when we patch the SAE reconstruction back into the model, so higher is better. Note, that it is masked LM loss for ESM-2 and causal LM loss for ProGen2, so the two numbers shouldn't be directly compared. *N conc.* counts the concepts (of 734) for which at least one feature reaches a test $F_1 \geq 0.5$, which is our detection threshold. The last three columns describe the features themselves. *Poly*, short for polysemantic, is the fraction of concept-mapped features that fire for more than one distinct concept, so a lower value means cleaner, more monosemantic features. *F/conc.* is the mean number of distinct features that detect each identified concept, which measures redundancy or feature-splitting, and here again lower is cleaner. *Interp* is the fraction of the whole dictionary that maps to any known concept.

The pretrained ESM-2 SAEs detect the most concepts, with ESM-2-650M reaching 298 at layer 9 and 271 at layer 24. Our ProGen2-large SAEs trail them,

Model	L	k / L_0	EF	Fidelity	N conc.	Poly (%)	F/conc.	Interp (%)
ESM-2-8M	4	106	32	92.4	121	7.3	17.8	19.4
ESM-2-650M	9	211	8	99.5	298	5.9	6.7	18.4
ESM-2-650M	24	121	8	98.9	271	4.1	6.5	16.6
ProGen2-S	7	k=50	16	96.1	277	5.8	10.5	16.8
ProGen2-S	7	k=96	4	95.4	162	8.2	3.7	13.4
ProGen2-L	9	5770	4	93.2	86	8.3	10.1	7.9
ProGen2-L	9	73	4	46.8	114	6.2	6.8	7.1
ProGen2-L	9	k=64	4	62.0	171	3.9	7.0	11.2
ProGen2-L	24	k=10	4	0.0	208	3.4	4.9	9.7
ProGen2-L	24	k=64	4	85.2	159	4.0	7.3	10.9
ProGen2-L	24	k=350	4	95.6	107	5.8	10.6	10.5

Table 4.2 Sparse autoencoders trained in this work: training hyperparameters, reconstruction fidelity, and feature-level interpretability metrics.

identifying up to 208 concepts for the layer-24 $k=10$ model. The one surprise is ProGen2-small, whose $k=50$ SAE identifies 277 concepts, essentially matching the much larger ESM-2-650M. Even though it identified more concepts than the large variant, it is likely due to having a large expansion factor and hence larger dictionary size. ESM-2-8M has by far the highest redundancy at 17.8 features per concept, meaning it splits each concept across many partly-redundant features, while the sparser ProGen2 models sit between 4.9 and 10.6. We will later explore the reason for this “redundancy” by looking at feature activations in a particular domain, and find that it happens because features are more narrowly specified, for example within domains, than the annotations make it seem. Polysemanticity stays low everywhere, mostly between 3 and 8 percent, so individual features rarely mix unrelated concepts in either family.

The ProGen2 rows let us relate the training settings to the resulting metrics, since here we control the model and vary only the SAE. The cleanest axis is the k -sweep on layer 24, where lowering k from 350 to 64 to 10 raises the concept count from 107 to 159 to 208. The same direction cleans up the features, and polysemanticity falls from 5.8 to 4.0 to 3.4 percent and redundancy from 10.6 to 7.3 to 4.9 features per concept. In other words, forcing each token to use fewer features makes those features more selective, which helps them line up with individual concepts. Fidelity moves the opposite way. The densest models reconstruct best but interpret worst, which is clearest in the two layer-9 ReLU SAEs. The near-dense one (L_0 around 5770) recovers 93.2 percent of the loss yet finds only 86 concepts, while the sparse one (L_0 around 73) recovers just 46.8 percent but finds 114 concepts. So fidelity, the metric that is cheap to watch during training, points the wrong way for interpretability, and a high-fidelity SAE is not the one we should necessarily pick for concept analysis.

Figure 4.1 breaks the same detection numbers down by concept type for nine of the SAEs. Each cell shows how many concepts of a given type the SAE identifies, shaded by the fraction of that type it covers. Domains dominate the annotation set, at 294 of the 734 concepts, and they dominate what every SAE finds, so the per-type domain row closely tracks the overall row at the top. ESM-2-650M leads here as well with 236 domains at layer 9, ProGen2-small is slightly worse at 203 (note that the ProGen2-small $k=50$ SAE was trained with a large expansion factor

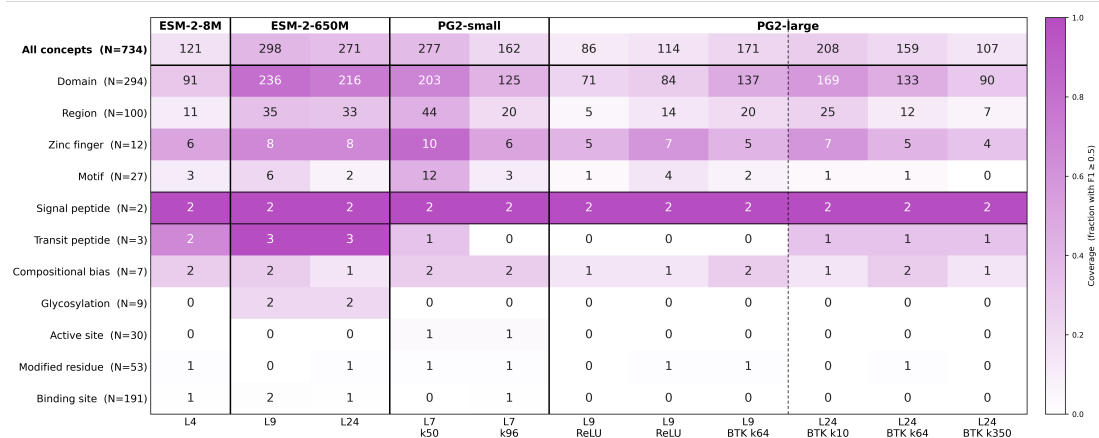


Figure 4.1 Per-concept-type detection coverage for nine of the trained SAEs. Each cell gives the number of concepts of that type identified at test $F_1 \geq 0.5$, shaded by the fraction of the type covered. Columns are grouped by model, with the ESM-2 SAEs from InterPLM on the left and our ProGen2 SAEs on the right.

making the comparison unfair), and the ProGen2-large models follow the same k ordering as before, from 169 domains at $k=10$ down to 90 at $k=350$. Regions, the next most common structural type, behave similarly, and ProGen2-small actually leads on them with 44 of 100.

A few concept types are detected well by every model. Both families reach full coverage on the two signal peptide concepts and strong coverage on zinc fingers, for instance 10 of 12 for ProGen2-small and 8 of 12 for ESM-2-650M. These are short, sequence-regular motifs that a causal model can pick up as readily as a bidirectional one. Other types expose a clear gap in favour of the masked model. ESM-2-650M covers all three transit peptide subtypes and even the small ESM-2-8M covers two, whereas every ProGen2 variant detects at most one, and two of them detect none. Glycosylation sites show the same pattern and are found only by ESM-2-650M. Both transit peptides and glycosylation depend on context that is not purely local, such as the organelle-specific cleavage context of a transit peptide, which the bidirectional model can attend to but the causal model, seeing only the preceding residues, largely misses.

At the other end, a group of types is essentially invisible to all of the SAEs. Binding sites are the single largest annotation category, at 191 concepts, yet almost none are detected by any model, and active sites and modified residues are nearly as bad. These concepts are often annotated on only one or two residues, so any feature that fires more often than there are annotations inevitably collects some false positives and may not cross the F_1 threshold. This is a limitation of the residue-level detection metric rather than of a particular model or SAE, since it affects the bidirectional and causal families equally. The masked model’s advantage is concentrated in the context-dependent types, namely transit peptides, glycosylation, and a few binding sites, while for short, locally-defined motifs the two families are close, and the sparsest residue-level concepts are not identifiable by either model family.

4.3 Feature Case Studies

The per-type coverage in the previous section made several concept groups look essentially undetected. Binding sites, active sites, and modified residues almost never reached our identification threshold of a test F_1 -per-domain above 0.5. In this section we look past the F_1 metric and inspect directly what individual features do, using the per-concept analysis (the concept explorer) and the protein zoom page from our dashboard. We pick a few illustrative concepts, namely ATP binding sites and nucleotide active sites, which are examples of rare concepts only annotated on a handful of residues. We also investigate the activations of features in the C2H2 Zinc finger, as an example of a short motif with some very conserved residues on several positions. The goal is not an aggregate score, but a demonstration that the SAE has features which localize even these rare concepts, and that our analysis makes them easy to find. All results below come from the ProGen2-large layer-24 SAE with $k=350$.

For each concept we retrieve the top five features using the annotation-enrichment mode, and for every feature, we sort the residues of each labeled protein by descending activation and ask how highly the annotated residues rank. We report the retrieval metrics defined in the methodology: hits@ k (the fraction of proteins in which at least one annotated residue lands in the top k), MRR (the mean reciprocal rank of the first annotated residue), and AUPRC, each averaged over the proteins carrying the concept. Alongside them we give the random baselines, i.e. the value each metric would take if we ranked the residues of the same proteins at random.

4.3.1 ATP Binding Site

The ATP binding site is annotated on 2,644 proteins from the test set, with a median of 8 annotated residues per protein, which corresponds to the P-loop motif, defined by the pattern `GxxxxGK[TS]`. We retrieved features with the annotation-enrichment mode, since the concept is too sparse for the F_1 -guided mode to be useful. Table 4.3 lists the top five features by enrichment together with their F_1 -per-domain and their retrieval metrics.

Feature	Enr	F1d	Precision	hit@1	hit@5	MRR	AUPRC
f7196	51.8×	0.24	0.22	0.36	0.68	0.524	0.284
f7514	42.2×	0.19	0.15	0.51	0.69	0.591	0.398
f5243	19.7×	0.21	0.37	0.53	0.71	0.611	0.208
f1111	18.4×	0.17	0.72	0.50	0.58	0.546	0.166
f683	16.6×	0.11	0.49	0.50	0.60	0.551	0.220

Table 4.3 Top five features for the ATP binding site concept, ranked by annotation enrichment score, on the ProGen2-large layer-24 $k=350$ SAE. *Enr* is the enrichment ratio, *F1d* is the test F_1 -per-domain, and *Prec* is the precision. Random baselines: hit@1 = 0.02, hit@5 = 0.09, AUPRC = 0.024.

None of these features would identify the concept using the 0.5 threshold, since the best F_1 -per-domain is only 0.248. The retrieval numbers show us that the model does indeed understand these concepts to a certain extent. The most

enriched feature, f7196, places an annotated residue in the single top-ranked position in 36% of the proteins and in the top five in 68%, against random baselines of 1.9% and 9.1%. Its AUPRC of 0.284 is more than ten times the baseline of 0.024. So a feature that the F1 metric ranks low, in fact points at the binding site with high reliability, and the *concept explorer* surfaces it immediately.

The table also shows why we include precision along with the enrichment ranking. Feature f7196 is the most enriched but has a precision of only 0.223, while f1111, which is four times less enriched, has the highest precision at 0.727. Enrichment compares the mean activation on annotated residues against the mean over all other residues, and that background mean is diluted by the millions of residues on which the feature is silent, so a high enrichment ratio does not bound the absolute number of false positives. Precision penalizes each false positive directly, so the two quantities pull apart. In practice this means the top enrichment features are a good place to start looking, but the per-feature precision and the retrieval metrics are what tell us how clean each one actually is.

Looking at the activation profiles in the *protein zoom* page makes the behavior clearer. On a typical P-loop protein, f7196 does not fire uniformly over the eight annotated residues but ramps up across them, from near zero at the start of the motif to its maximum on the last two residues, and is silent almost everywhere else in the sequence. The top five features are also partly complementary rather than redundant. Figure 4.2 shows the screenshot from the *Protein zoom* page which plots the activation profile of the 5 features in a sequence (Q0HYN8), zoomed into the P-loop region. We see that each of the 5 features fires in a slightly different pattern, for example feature f1111 fires only on the conserved lysine, feature 7514 fires on all 8 residues in the P-loop, and feature 7196 fires on the second half of it and two residues after.

We also noticed that the annotation length itself is heterogeneous, ranging from a single labeled residue to the eight-residue P-loop in the examples we inspected, and some ATP sites elsewhere in the data are labeled far more broadly. Since the ligand is the same, we would naively expect the sites to be of comparable size, so part of this spread may be annotator inconsistency rather than biology.

4.3.2 Nucleophile Active Site

The nucleophilic active site is an even harder case. It is annotated on 891 proteins in our test set, and crucially the annotation is a single catalytic residue, so the median number of positive residues per protein is exactly one. Table 4.4 gives the top five features by enrichment.

Feature	Enr	F1d	Prec	hit@1	hit@5	MRR	AUPRC
f5222	55.5×	0.112	0.093	0.37	0.50	0.429	0.400
f7293	26.7×	0.012	0.006	0.25	0.42	0.327	0.279
f6598	23.2×	0.014	0.007	0.05	0.15	0.108	0.091
f1105	21.4×	0.007	0.004	0.03	0.05	0.055	0.451
f2492	14.1×	0.007	0.004	0.07	0.31	0.174	0.116

Table 4.4 Top five features for the nucleophile active site, ranked by annotation enrichment, on the ProGen2-large layer-24 $k=350$ SAE. Columns as in Table 4.3. Random baselines: hit@1 = 0.003, hit@5 = 0.014, AUPRC = 0.003.

Protein Zoom

Run several SAE features on one protein and compare their per-residue activation profiles.

[Q0HYN8](#) — L=361 (8 residue(s) annotated for 'Binding site_ATP')

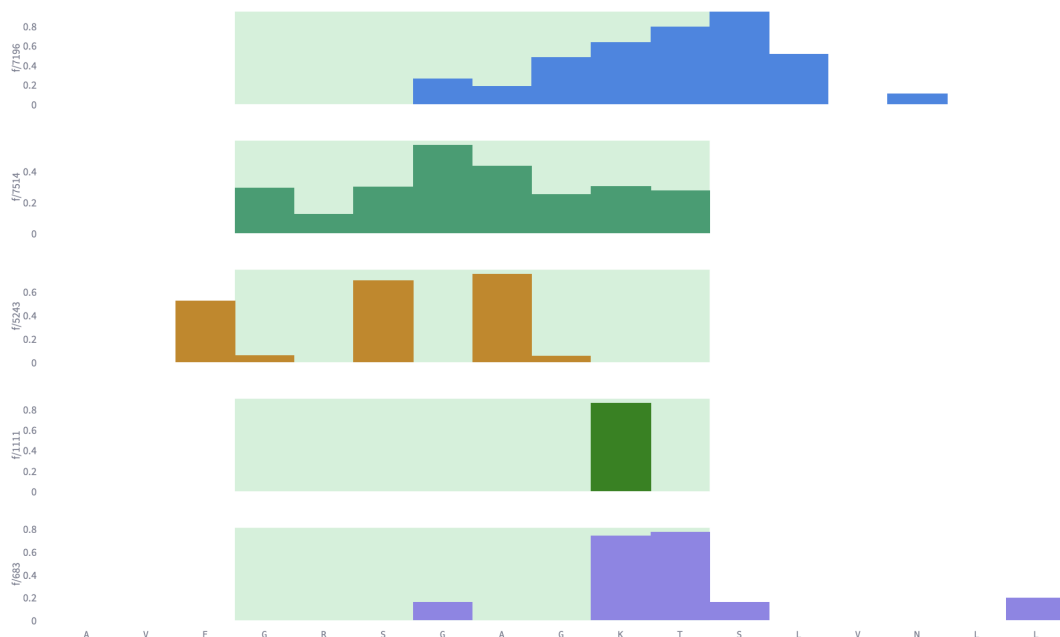


Figure 4.2 Activation profiles of the top 5 features by enrichment for the ATP binding site concept. Computed for sequence with UniProt ID: Q0HYN8

Here F1 and F1-per-domain coincide, because with a single annotated residue per protein the per-domain recall reduces to the global recall. The best feature, f5222, has an F1 of only 0.112, which is far below our threshold and would place this concept firmly in the undetected group. Yet the same feature ranks the single catalytic residue first in 37% of the proteins and reaches an AUPRC of 0.400, against random baselines of 0.3% in both cases. That is more than 100 times better than randomly picking out one residue in a protein of median length 370. This is a clear example that the F1 threshold undersells a feature that is, in retrieval terms, performing quite well. The remaining four features are much weaker and mostly fire too broadly, which shows again that enrichment on its own is only a first filter and the retrieval metrics are needed to separate the one strong feature from the rest.

4.3.3 C2H2 Zinc Finger

Our last example is different in character. The C2H2-type zinc finger is a short, well-conserved structural motif, and it is annotated densely, with a median of 114 annotated residues per protein because these proteins typically carry several copies of the domain. This is a concept the F1 metric handles well, so we retrieved its features with the F1-guided mode instead. Note that for this concept, we searched for the features in the ProGen2-large layer 24 SAE trained with $k=10$, rather than $k=350$ from the previous sections. We show the features and their retrieval metrics in table 4.5. In case of larger annotated regions such as motifs and domains, the retrieval metrics like hits@ k and MRR make less sense than in the small binding and active sites, but we still report them for completeness.

Feature	F1d	Prec	hit@1	hit@5	MRR	AUPRC
f4720	0.927	0.933	0.96	0.97	0.968	0.577
f4223	0.926	0.877	0.97	1.00	0.986	0.600
f5994	0.925	0.972	0.89	0.94	0.907	0.662
f9415	0.924	0.975	0.94	0.96	0.950	0.576
f3851	0.867	0.998	0.94	0.96	0.954	0.568

Table 4.5 Top five features for the C2H2-type zinc finger, ranked by F1-per-domain, on the ProGen2-large layer-24 $k=10$ SAE. Columns as in Table 4.3, without the enrichment ratio. Retrieval columns are averaged over the 108 labeled proteins. Random baselines: hit@1 = 0.231, hit@5 = 0.732, AUPRC = 0.318.

All five features reach an F1-per-domain above 0.86 and a precision above 0.87, and all of them place an annotated residue at the very top in more than 89% of the proteins. Unlike the nucleophile case, where a single protein usually triggered only one of the top features, here the top five features are all active on the same proteins and complement each other, with each one firing on a different part of the repeated domain. We attribute this to the domain being short and strongly conserved, so the SAE can “afford” several precise features that together tile the motif rather than a single noisy one. We show this in Figure 4.3, an expanded protein zoom view of one zinc-finger protein with all five features on the annotated domains. For example, the fourth depicted feature (f9415) activates only on the two conserved histidines in the motif. We return to this concept later, when we attempt to steer ProGen2 towards generating zinc-finger sequences.

Taken together, these three cases show the two ends of what our tools can recover. For a densely annotated, conserved domain like the zinc finger, the ordinary F1-guided view already returns a set of clean, complementary features. For sparse residue-level concepts like the ATP and nucleophile sites, where the F1 metric alone would report weak signals, the enrichment mode still surfaces features that localize the annotation far above chance, and the protein zoom page lets us confirm that they fire on the annotated region rather than merely somewhere in the protein. The features are there in all three cases. What changes is how much work it takes to find them and how much the aggregate metric hides.

4.4 Steering Experiments

We performed several steering experiments. In each one we validate the outputs with a publicly available tool depending on the task. Some of the tasks are simple and meant only to establish that the model can be steered at all, while others are more difficult, and we report also the failed ones.

All experiments use ProGen2-large with the intervention placed at layer 24. We chose this depth because the corresponding layer in ESM-2 had the most concepts identified, and we kept it as the default throughout this chapter. For the experiments below we compare two SAEs trained on this layer, same as in prior sections ($k=10$ and $k=350$). We were curious whether these two different SAEs can both be steered, and how the steering differs between them, given the difference in reconstruction fidelity and number of concepts identified.

Protein Zoom

Run several SAE features on one protein and compare their per-residue activation profiles.

[P18713](#) — L=197 (162 residue(s) annotated for 'Zinc finger_C2H2-type')

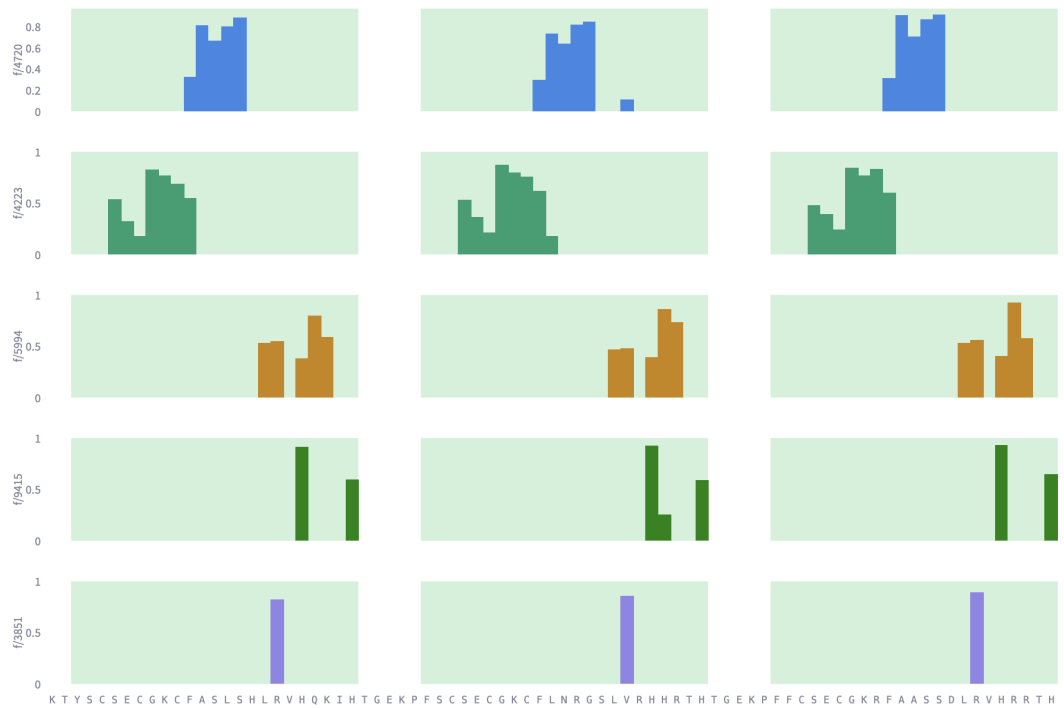


Figure 4.3 Protein zoom view for a C2H2 zinc-finger protein. Green regions mark the annotated zinc-finger domains, and the stacked bar graphs give the activation of each of the five features from Table 4.5 across the sequence.

4.4.1 Disorder and Compositional Bias

SwissProt annotates disorder and compositional bias automatically with MobiDB-lite. The tool first marks the disordered regions of a sequence and then, within those regions, potentially, regions of compositional bias, meaning an unusually high frequency of one amino acid or one class of amino acids, such as glycine-rich or highly acidic regions.

Disorder and compositional bias are convenient first targets for steering. Only one concept in this category reached an F1 above 1/2, but a low F1 does not rule out a feature being steerable, and the property itself is simple, since it just means enriching the output for certain amino acids. We could of course obtain the same enrichment trivially by biasing the model’s output probabilities, but the point of steering is to intervene on SAE activations in the residual stream between hidden layers of the model, and observe whether the downstream layers of the model still understands that intervention and the generated sequence satisfies the property.

To know what disorder looks like by default, we ran 500 randomly chosen test sequences from UniRef50 through MobiDB-lite. 30.8% of them carried any disorder label, and 9.6% of all residues were labeled disordered. The background frequencies were 11.3% for acidic residues (D+E), 6.9% for glycine, and 5.3% for proline. We also need a baseline for the model’s own unconditioned behavior, so we generated 500 sequences of at most 350 tokens, without any steering, both with no prompt and with each of the 6 prompts introduced later. With no prompt and no steering, 16.5% of residues were disordered. This is higher than the 9.6% seen in real UniRef50 sequences, possibly because disordered stretches tend to sit at the start of a sequence. These unsteered sequences were scored with MobiDB-lite in exactly the same way as the steered ones.

Feature Selection

We tested three properties: general disorder, proline content, and acidic (D+E) bias. Proline (P, Pro) content is the simplest, being a single residue. Acidic bias covers the two acidic residues, aspartic acid (D, Asp) and glutamic acid (E, Glu). General disorder is the broadest, since the compositional-bias labels are all subsets of the disorder labels. The SAEs contain features for each of these bias types, and for the other bias types too, but we decided to only test these three.

For disorder we first ask which features to steer and whether a feature’s retrieval quality predicts how well it steers. We measure steering success as the fraction of generated residues that MobiDB-lite places inside a disordered region. From each SAE (k=10 and k=350 on ProGen2-large L24) we take the five features with the highest F1 and the five with the highest F1-per-domain, and report their indices, precision, recall, recall-per-domain, F1, and maximum observed activation in table 4.6. We can see that the two rankings are quite different. A feature that ranks high by F1 is often not high by F1-per-domain, and the other way round.

We sweep clamp values for each feature to find a useful range, shown in figure 4.4. The grids differ between the two SAEs. The k=10 features have larger maximum activations, but the SAE has only ten active features in total, so clamping one of them to ten times its maximum, the value of k, would overwhelm everything else in the residual stream, therefore we keep the multiples lower than for k=350. From the sweep we decided to use a clamp of 3x for k=10 and

8x for $k=350$ for all features in that SAE for simplicity. The last column of table 4.6 shows the percentage of residues labeled as disordered when we steer the particular feature. Some of the highest-F1 features perform worse than the unsteered baseline, while some lower-ranked features drive disorder all the way up to 100%.

Feature	Rk _{F1}	Rk _{F1d}	P	R	R/d	F1	F1d	Max act.	Dis%
<i>Baseline (unsteered, both SAEs)</i>									16.5
<i>k = 10 SAE (Dis% at clamp 3×)</i>									
f4527	1	43	0.28	0.24	0.35	0.26	0.31	18.19	9.0
f6477	2	30	0.40	0.18	0.28	0.25	0.33	18.76	97.6
f3226	3	56	0.26	0.15	0.28	0.19	0.27	26.96	37.0
f9570	4	39	0.66	0.08	0.21	0.14	0.32	24.04	100.0
f2897	5	22	0.26	0.09	0.80	0.14	0.40	12.53	59.4
f10207	55	1	0.62	0.04	0.54	0.07	0.58	17.37	63.4
f1105	18	2	0.51	0.06	0.66	0.11	0.57	13.00	87.3
f8829	11	3	0.48	0.07	0.70	0.12	0.57	18.31	100.0
f7193	39	4	0.50	0.05	0.63	0.09	0.56	16.57	98.1
f4443	9	5	0.48	0.07	0.67	0.12	0.56	17.17	91.5
<i>k = 350 SAE (Dis% at clamp 8×)</i>									
f5784	1	26	0.35	0.39	0.83	0.37	0.49	9.72	95.7
f4838	2	83	0.41	0.27	0.45	0.32	0.43	6.80	6.4
f6828	3	3	0.57	0.18	0.57	0.28	0.57	9.16	39.2
f6571	4	70	0.37	0.21	0.54	0.27	0.44	10.67	16.1
f3957	5	33	0.38	0.19	0.65	0.26	0.48	11.64	34.7
f357	10	1	0.59	0.14	0.60	0.23	0.60	6.59	60.1
f8138	60	2	0.50	0.09	0.73	0.15	0.60	11.67	9.9
f9396	31	3	0.51	0.08	0.71	0.13	0.59	9.38	10.2
f1458	44	4	0.54	0.11	0.64	0.18	0.58	7.64	13.5
f2308	31	5	0.48	0.09	0.72	0.15	0.58	8.13	26.5

Table 4.6 Retrieval metrics and steerability for all tested disorder features across both SAE checkpoints ($k=10$ and $k=350$). Rk_{F1} and Rk_{F1d} are global ranks by F1 and F1-per-domain across all features for **Region_Disordered** concept. Within each SAE block, the upper group was selected by F1 and the lower by F1d.

We also looked at what kind of disorder the steering produces. Disorder can arise from a generally low content of hydrophobic residues, but also from long single-residue repeats, for example a poly-proline stretch, which is disordered for a different reason. For each steered feature we highlight the resulting compositional bias by type in figure 4.5. Most features give a profile close to the baseline, with no strong bias, but a few do not. For instance, feature f1105 of the $k=10$ SAE, generates proline-rich sequences (73.7% proline) that are degenerate and unnatural yet still labeled as disorder. We treat this as acceptable, since our aim is not to produce realistic proteins but to satisfy the target property, so it is fine for the outputs to be “caricatures” of real sequences.

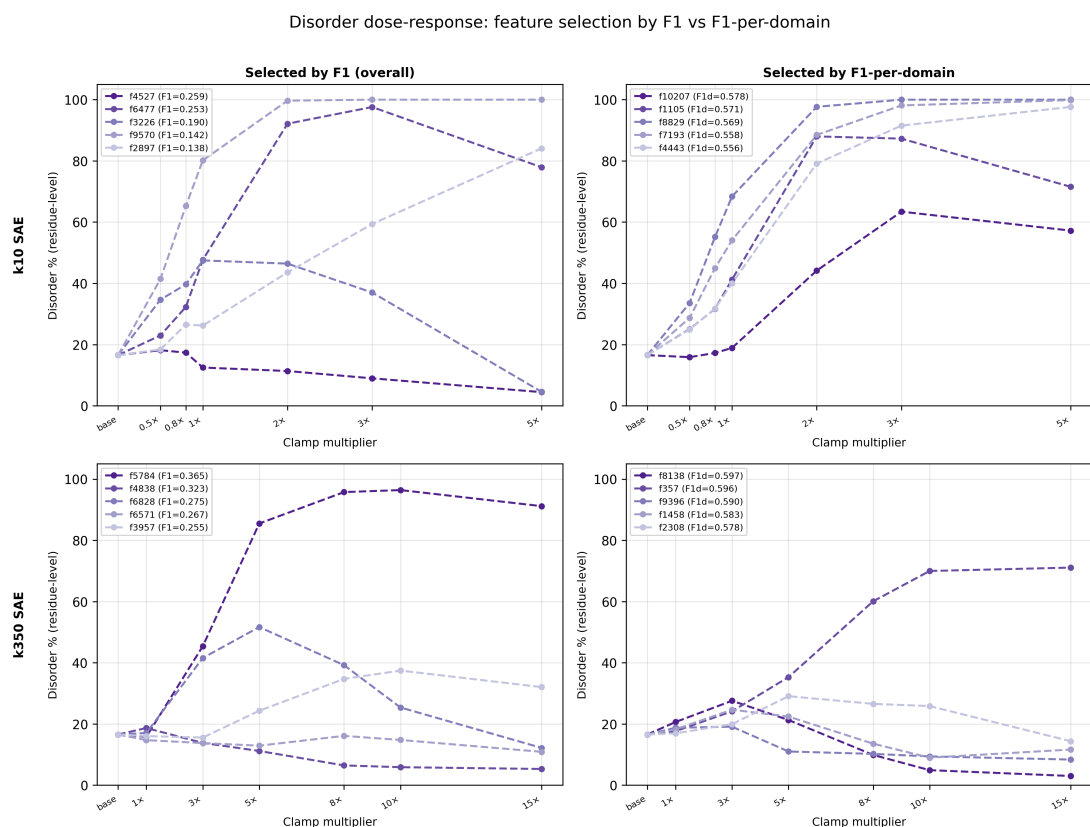


Figure 4.4 Achieved percentage of disordered residues for various features and clamp values. Top two plots use k=10 SAE and bottom two use k=350 SAE. The grid over which clamp values are tested is different for each SAE, both starting with the unsteered baseline.

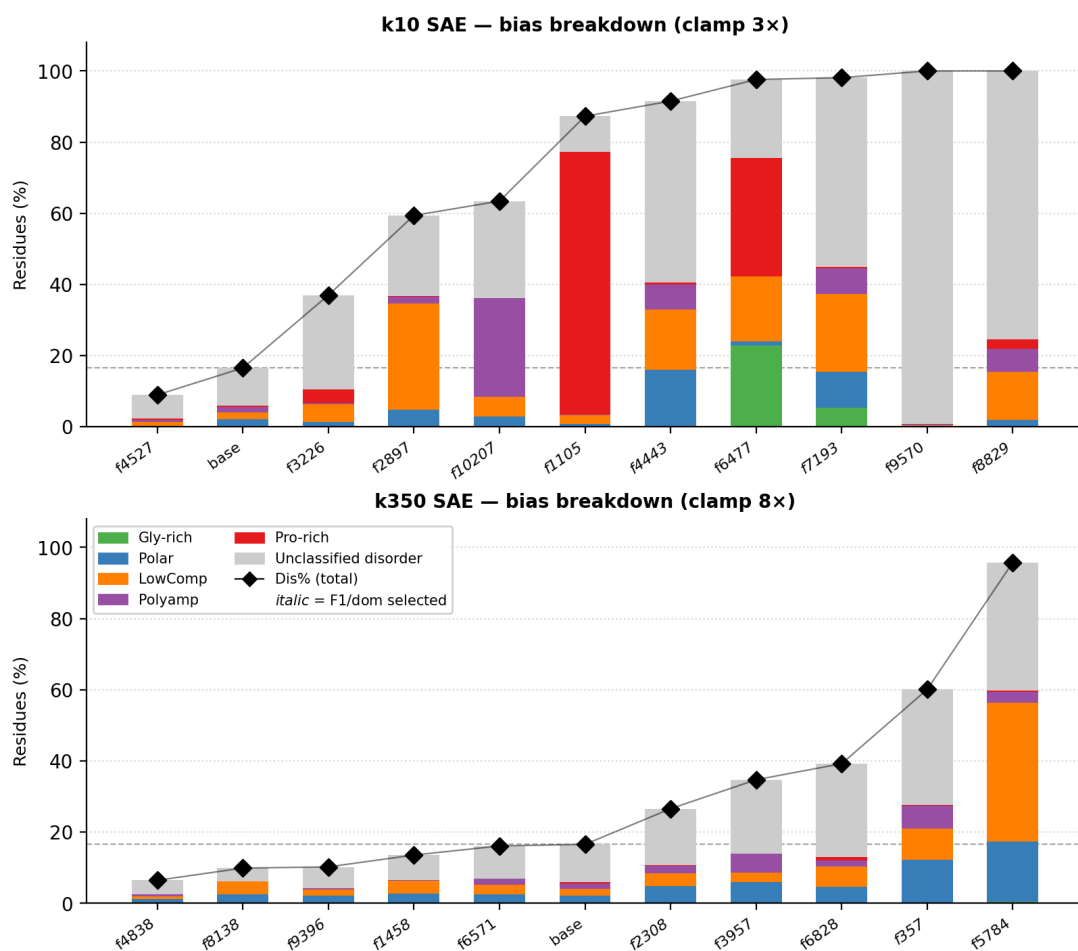


Figure 4.5 MobiDB-lite compositional bias breakdown for disorder-associated SAE features at fixed clamps (k10 @ 3×, k350 @ 8×), showing the fraction of generated residues attributed to each bias sub-type (stacked bars) alongside total disorder fraction (black diamonds); the dashed line marks the unsteered baseline. Features are sorted by disorder fraction; italic labels indicate F1/dom-selected features.

Steering for other properties

We would also like to experiment with steering directly for acidic (D+E) and proline (P) content. For each property we steer the feature with the highest F1-per-domain, over the same clamp grid as before. Here steering is also effective, with success approaching 100%, and we again fix the clamp at 3x for $k=10$ and 8x for $k=350$. Table 4.7 reports not only the targeted property but also the off-target ones, the fraction of sequences with any disorder region, the mean number of distinct amino acids, and the mean longest single-residue run. These off-target columns show how degenerate the strongest steering becomes. Proline steering on the small SAE drives the mean longest run to 32 and leaves only 11 distinct amino acids per sequence, far below the baseline. The *NPoly* column is MobiDB-lite’s own label for acidic compositional bias regions, so *NPoly*% is always at most *Dis*%(r), since regions of compositional bias are subsets of IDRs. The D+E column is the raw acidic fraction we compute directly from the sequence.

Feature	Type	Dis%(r)	Dis%(s)	NPoly%	D+E%	P%	NLL	UAA	MaxRun
—	baseline	16.5	36.7	2.3	11.4	6.1	2.48	19.0	3.1
<i>k</i> = 10 SAE, clamp 3×									
f7220	Acid	100.0	100.0	100.0	85.8	0.4	1.43	11.2	9.0
f9644	Pro	91.8	100.0	0.0	0.4	80.9	1.07	11.1	32.4
f10207	Dis	46.6	77.3	10.2	14.2	4.0	2.50	16.5	3.4
<i>k</i> = 350 SAE, clamp 8×									
f4374	Acid	32.2	69.3	46.5	29.6	3.0	2.82	18.8	3.5
f8626	Pro	41.1	68.5	0.8	9.5	12.3	2.60	18.8	3.3
f357	Dis	47.8	75.8	4.7	9.3	4.2	2.70	18.8	3.0

Table 4.7 Generation results ($n = 128$ per condition, prefix 1M). Dis%(r) = mean fraction of residues in consensus disorder; Dis%(s) = fraction of sequences with any disorder call; NPoly% = MobiDB-lite compositional bias denoting acidic residues; D+E% and P% = mean residue-level amino acid composition (manually computed); UAA = mean unique amino acids per sequence; MaxRun = mean longest consecutive run of one amino acid.

Steering after a prompt

So far the model generated from scratch. We now use a prefix (prompt) and ask whether steering can still impose the target property once the model has context to continue. We use 6 domains with conserved but structurally diverse folds, with the following canonical names, UniProt codes, length and short description:

- Ubiquitin, P0CG48, 76 residues, $\alpha + \beta$ fold, a well-characterised globular domain,
- Src SH3 domain, P12931, 63 residues, all- β fold, a small protein-interaction module,
- Antennapedia homeodomain, P02833, 61 residues, all- α fold, the helix-turn-helix DNA-binding domain of a homeotic transcription factor,

- GB1, the B1 domain of streptococcal protein G, P06654, 56 residues, $\alpha + \beta$ fold,
- CheY, P0AE67, 129 residues, $(\beta\alpha)_5$ flavodoxin-like fold, a bacterial chemotaxis response regulator,
- HMGB1 box A, P09429, 75 residues, all- α fold, an HMG-box DNA-binding domain.

Without steering we record how often each prefix is continued into a disordered region, and with steering we ask whether the intervention is enough to override that tendency. This is harder than the no-prefix case. With no context the model was easy to push, but a prefix carries what the model “remembers” from training, and we want to see whether the steering signal is “stronger” than that memory. The protocol is otherwise unchanged, with the same clamp values, the same features, the same maximum length, and $n=128$ sequences per prefix and per steered feature. The results are shown in table 4.8.

Prefix	<i>L</i>	Acidic (D+E%)		Pro (P%)		Disorder (Dis%)	
		Base	Steer	Base	Steer	Base	Steer
<i>k10 SAE (f7220 Acidic 3×, f9644 Pro 3×, f10207 Disorder 3×)</i>							
None (1M)	2	11.4	85.8	6.1	80.9	16.5	46.6
Ubiquitin	76	12.8	89.8	4.4	91.9	4.3	38.6
Src SH3	63	11.9	82.2	5.8	83.0	13.7	64.3
Antp HD	61	12.6	88.6	6.5	90.1	50.0	75.1
GB1 B1	56	15.5	88.2	4.1	84.4	9.4	58.0
CheY	129	13.6	62.6	2.7	78.8	0.5 [†]	38.6
HMGB1	75	26.8	87.6	6.8	87.1	49.6	81.7
<i>k350 SAE (f4374 Acidic 8×, f8626 Pro 8×, f357 Disorder 8×)</i>							
None (1M)	2	11.4	29.6	6.1	12.3	16.5	47.8
Ubiquitin	76	12.8	31.1	4.4	8.3	4.3	22.6
Src SH3	63	11.9	29.9	5.8	11.3	13.7	47.6
Antp HD	61	12.6	31.4	6.5	7.8	50.0	47.0
GB1 B1	56	15.5	28.7	4.1	7.6	9.4	24.1
CheY	129	13.6	27.0	2.7	5.7	0.5 [†]	20.9
HMGB1	75	26.8	30.2	6.8	9.2	49.6	66.0

Table 4.8 Generation after a prompt, with SAE feature steering ($n=128$). Each condition shows the baseline and steered value for its target metric only: D+E% for Acidic, P% for Pro, disorder fraction for Disorder. k10 clamp 3 $\times$, k350 clamp 8 $\times$. [†] CheY baseline triggers early EOS after the 129-aa prefix; steered runs override termination.

4.4.2 Transit Peptides

The next condition we steer towards are transit peptides, also called target peptides. These are short sequences, typically around 30 residues, that sit at the N-terminus of a protein and are followed by the rest of the protein. Their purpose is to direct the protein to its destination compartment inside the cell, or out of the cell entirely. Once the protein arrives, the transit peptide is cleaved off and the rest of the protein carries out its normal function.

Transit peptides are an appealing target for steering for two reasons. First, they are generally not conserved, which means their exact sequence varies a lot between proteins, so the model has a lot of freedom in how it satisfies the condition. Second, they sit at the very N-terminus, so for a causal language model such as `progen2` no additional prefix is required. We can steer from the first generated residue onward, unlike a condition that only makes sense in the middle of a protein.

The setup is similar to before. We use the `k=350` SAE trained on layer 24 of `progen2-large` and prompt the model only with `1M`. For each experiment we generate 512 sequences of up to 50 residues, which is enough since transit peptides are short. We intervene by clamping one or more feature activations to a chosen value and decoding the hidden state back into the residual stream (`with_error` steering mode).

To decide whether a generated sequence actually contains a transit peptide, we pass the whole batch as a FASTA file to TargetP 2.0. For each sequence TargetP returns a prediction over three classes: `noTP` (no targeting peptide), `SP` (signal peptide) and `mTP` (mitochondrial targeting peptide), together with the probability of each class and, when a peptide is found, the position of the cleavage site that marks the end of the transit peptide region. The prediction is the argmax of the three probabilities. We are not interested in the specific compartment, so we merge `SP` and `mTP` into a single positive class, which we refer to as `TP`, and count a sequence as a success whenever the prediction is not `noTP`. Our metric is the percentage of such sequences across the generated set, which we denote `%TP`.

Finding relevant features

The first task is to find features worth steering. The strongest candidates by F1 were feature 5900, 7356, and feature 3884. Note, this experiment was performed before we implemented the concept analysis mode and dashboard pages, so we used a more ad-hoc method for identifying relevant and most importantly steerable features. For example, feature 7356 had the highest F1 of all but steered poorly, which shows that a feature being predictive of a concept does not mean it causally drives the generation of that concept.

To find features that are actually causal, we used iterative anti-target analysis: for a given batch of generated sequences, we split them into the `TP` and `noTP` groups according to TargetP, embed them through layer 24, encode them with the SAE and compare the mean and maximum activation of every feature between the two groups. Features that fire much more often in the `TP` group are candidates for positive steering, and features enriched in the `noTP` group are candidates for negative steering. Running this on the output of an early feature combination surfaced feature 8227, which fired in 94.2% of `TP` sequences but only 51.9% of

noTP sequences. Adding it to the combination raised the %TP substantially. A second round on the improved batch found a further pro-TP feature, feature 1323, and two features enriched in the remaining noTP sequences, features 8839 and 3266.

This kind of comparison of features active on sequences satisfying some constraint versus features activating on sequences that do not satisfy it, was the inspiration behind the annotation enrichment analysis mode in the *concept explorer*.

Steering results

We first steered one feature at a time. Each candidate gave a noticeable improvement over the unsteered baseline of 31.8% at its best clamp value, as summarised in the top block of table 4.9. Feature 8227 was the strongest on its own, reaching 67.6%. Note, that the gain is not predicted by the F1 score that first flagged the features: feature 7356 had the highest F1 yet gives the smallest improvement.

Steering several features at once improved things further. Combining the three strongest positive features, f5900:5 + f3884:10 + f8227:7, reached 75.0%, already more than double the baseline.

We also looked for features that work against transit peptides and tried to use them to push the model away from generating them. Steering the anti-target features on their own, with no positive features, kept the %TP at baseline or below, which confirms that the positive features carry the functional effect. The more useful trick was to steer these anti-features with a negative clamp value. Negative steering is not just suppression, but setting the feature to a negative value, which means a multiple of the feature’s SAE decoder row is subtracted from the hidden state instead of added, actively reversing the feature’s contribution to the forward pass. Since the anti-features push the model towards noTP, subtracting their direction should push it towards TP, and indeed it did.

Note, that not every anti-target feature helped this way. Features 8643 and 1422 lowered the rate when steered negatively, while features 8839 and 3266 raised it. We assume this reflects the difference between features that are genuine drivers of a non-TP trajectory and features that are only a downstream consequence of one.

The best configuration combined both directions: the three positive features together with the two useful anti-features steered negatively, f5900:5 + f3884:10 + f8227:7 + f1323:2 + f8839:−1 + f3266:−1. This reached 78.7% TP against the 31.8% baseline, a 2.5-fold increase. Table 4.9 shows the full progression. We also include the opposite extreme, which involves suppressing the strongest positive feature, or negating all of the pro-TP features at once, drives the rate well below baseline, down to 4.5%.

Sequence quality

A steering result is only meaningful if the generated sequences are still protein-like. It is genuinely hard to prove that a sequence is a “good” protein, but we can at least check that our outputs are not obvious garbage, using three cheap proxies.

Steering configuration	%TP	%frag	NLL
Unsteered baseline	31.8	3.5	2.796
<i>Single features (best clamp)</i>			
f8227:10	67.6	3.7	3.06
f3884:10	59.0	6.4	2.98
f5900:10	45.1	1.4	3.10
f7356:2	34.6	2.9	2.89
<i>Combinations</i>			
f5900:5 + f3884:10 + f8227:7	75.0	5.9	3.07
+ f8839:−1 + f3266:−1	77.5	4.7	3.07
+ f1323:2 + f8839:−1 + f3266:−1 (best)	78.7	4.3	3.07
<i>Suppression and negative steering</i>			
f8643:−2 + f1422:−2 + f3266:−1 (negative only)	30.9	2.5	2.96
f8227:−2	20.9	2.5	2.86
f8227:−5	8.6	0.2	2.99
all pro-TP features negated	4.5	1.4	3.13

Table 4.9 Target peptide steering configurations grouped into single features, combinations, and suppression. Within the combinations block the indented rows build on the three-feature positive combination above them. %TP is the percentage of sequences predicted as TP (n=512 per experiment) %frag and NLL (negative log likelihood) are sequence-quality proxies.

The first is the negative log-likelihood (NLL) of each sequence under the unsteered ProGen2-large. A low NLL means the base model finds the sequence plausible, while a high NLL means it lies far outside the training distribution. The second is the fragment flag from TargetP, which marks sequences that are too short to be a real protein. The third is the mean sequence length, since a model that starts emitting the end-of-sequence token too early is a sign that steering has broken generation.

Across all of our functional steering conditions the NLL stayed in a narrow band around the baseline, roughly 2.8 to 3.1, fragment rates stayed below 6%, and mean length held near the generation cap. In other words the sequences that scored well on TargetP were no less plausible to the base model than the unsteered ones, which argues that steering is changing the function of the sequences rather than degrading them. As a contrast, pushing a single feature far outside its training range does break generation. Steering feature 5900 at 30 times its maximum activation raises the NLL above 5 and collapses the mean length below 4 residues. This marks where the useful steering range ends, but within that range the quality proxies stay flat.

4.4.3 Domain and Motif Generation

The targets so far, disorder, compositional bias, and transit peptides, were all either compositional properties or N-terminal signals. We now turn to a harder class of target, the internal structural domains and sequence motifs. These are appealing because they are exactly the kind of concept the SAE identifies most reliably, since domains dominate the identified concepts. We tried three: the

C2H2 zinc finger motif, the Sushi (CCP) domain, and the EF-hand calcium-binding domain. We selected the first two examples because they are relatively short domains/motifs (up to 60 residues) with the only constraints for generation being conserved cysteines in some positions. The EF-hand domain is structurally conserved as well as sequentially conserved, but only 12 residues. As before we generate from a bare methionine (M) prefix on the ProGen2-large layer-24 $k=350$ SAE, clamp one or more feature activations to a multiple of their training maximum, and generate 512 sequences per condition. We validate the outputs with `ps_scan`, which matches each sequence against the family’s PROSITE entry (PS00028, PS50923, and PS00018 respectively). Across all three, steering produced no hits. The rest of this section reports what we tried and our hypotheses for why the approach breaks down, which differ between the geometric, cysteine-constrained motifs (zinc finger and Sushi) and the EF-hand.

Zinc finger motif (C2H2)

The C2H2 zinc finger is the motif we examined earlier when we looked at its features. As we showed earlier, it was not represented by a single “zinc finger” feature but by several, each firing on a different part of the repeated finger. Its signature PS00028, `C-x(2,4)-C-x(3)-[LIVMFYWC]-x(8)-H-x(3,5)-H`, needs two cysteines and two histidines at fixed spacings. We steered the strongest features by F1-per-domain (f5530, f5134, and f6758), both singly and combined and at clamp values up to ten times their training maximum, and every configuration gave 0 hits out of 512.

The amino acid composition explains why. The features do raise histidine, from 2.1% at baseline to 6.3% for the four combined, but they barely move cysteine, which stays near 1% against the roughly 3% of natural zinc-finger proteins, so the `C-x(2,4)-C` pair never gets generated. The features encode the zinc-finger context but not the placement of the rare cysteines it depends on. Even when we seed the cysteine pair ourselves with a short `MCKPYC` prefix, the same features barely help, lifting the hit rate only from 1.17% to 1.56%.

Sushi domain

The Sushi domain, also called the CCP module, is a roughly 60-residue fold held together by two disulfide bonds between four conserved cysteines at the spacing `C-x31-C-x16-C-x14-C` (PS50923). It fails for a related reason. The features that score highest for it by F1-per-domain fire very broadly, in 70 to 96% of proteins, unlike the transit peptide features that fire in only about 10%. Such high-fire-rate features are “recognition” features that light up whenever the model processes beta-sheet-rich, disulfide-containing content, rather than “causal” features that drive it when clamped, and clamping them in fact lowers cysteine content rather than raising it. Every single-feature and combined condition gave 0 hits out of 512. As with the zinc finger the model can still produce the domain when we prompt it instead of steering, since a 34-residue prefix containing the first two conserved cysteines continues into a valid Sushi domain 95% of the time.

EF-hand domain

The EF-hand is a helix-loop-helix calcium-binding domain whose 12-residue signature PS00018 is compositional and, unlike the previous two, cysteine-free, needing only common residues such as aspartate, asparagine, and glycine. It is a useful control, since if the earlier failures were only about placing rare cysteines, the EF-hand should be easier. Its features (f9527, f617, and f8208) still produce 0 hits from a methionine prefix, against a unsteered baseline of 0.78%. The features fire at variable internal positions inside a helix-loop-helix context, and clamping them from position 1 tells the model that an EF-hand is present but not where along the sequence to place the loop.

5 Discussion

We acknowledge some limitations in our approach and think about directions that future works could focus on.

5.1 Limitations

There are a few limitation and simplifications we did in this work:

- We trained and analyzed SAEs on only a handful of layers (layers 9 and 24 of 32 for ProGen2-large, layer 7 of 12 for ProGen2-small), not a full sweep across depth.
- Most of our detailed analysis, including the case studies and the steering experiments, focuses on a single checkpoint, the layer-24 SAE of ProGen2-large, so those findings are tied to that specific setting.
- We trained on considerably fewer sequences than the original InterPLM publication (500k vs 5M), due to storage constraints, which we do not believe changed our results much, but did not verify.
- Most of our ProGen2 SAEs use the BatchTopK activation function, while the pretrained ESM-2 SAEs from InterPLM mostly use ReLU with an L1 penalty.
- Our quantitative evaluation is limited by existing annotations, so it can only measure how well a feature recovers something already known. The LLM annotator approach (autointerp) would help this issue, but it still largely depends on the presence and quality of existing annotations and having a budget for it.

5.2 Future Work

Several open questions and possible extensions follow directly from the limitations above.

- Compare how *steerability* changes with depth, and more generally which layer is best suited for a given property.
- Our comparison between SAE checkpoints relies mostly on how many concepts each one identifies, which is a fairly global metric. A more detailed comparison, looking directly at how the learned features themselves differ between checkpoints would be interesting.
- More broadly, we think there is room to develop better metrics for quantifying both interpretability and steerability than the ones we used here. Especially during training of SAEs, there is no good proxy for interpretability, fidelity even had negative correlation with number of concepts identified.

- We mostly looked at the highest-activating examples of a feature, the clearest cases of the property it tracks, but it is less obvious what a feature firing at only 30% of its maximum means, whether the protein satisfies the property to a smaller degree, or the feature is only active at that level because of the surrounding context set by other active features. Metrics like polysemanticity address this partially, and a fuller answer would need closer, qualitative analysis of individual features.
- It would also be interesting to apply this pipeline to a structure-aware model, for example one trained on the Foldseek [54] 3Di alphabet, rather than on sequence alone.

Conclusion

In this thesis we set out to interpret a generative protein language model and to also control its outputs. We had three goals: to compare the interpretability of a generative and a masked protein language model, to extend the InterPLM tooling to make finding interpretable features easier, and to attempt steering a generative model towards a desired property using the features we found. We achieved all three.

We successfully replicated the InterPLM pipeline for ProGen2, a generative protein language model, which required adapting several parts of the pipeline that had so far only been used with a masked model such as ESM-2. Along the way, we made the pipeline considerably more computationally efficient and fixed several bugs in the original implementation, including one that had silently prevented an entire category of concepts, binding sites, from ever being searched for.

Using this pipeline, we compared the two model families and found some differences in what their features represent, particularly for properties that depend on context from both directions of a sequence, which only the masked model can use. We also built new ways of exploring the trained sparse autoencoders, letting a user start an analysis from a concept or a protein of interest rather than only from an individual feature, extending the original InterPLM dashboard. We found these additions practical in our own work, and believe they can be useful in future interpretability research on protein language models.

Finally, we attempted to steer the generative model towards several target properties by manipulating its SAE features during generation. This succeeded for compositional properties, such as disordered regions compositional biases within them, and for generating transit peptides, showing that this kind of steering is possible for a generative protein model, which was not done before. Steering towards generating whole structural domains did not succeed.

Overall, we consider the goals of this thesis fulfilled. We hope this is a useful starting point for future work, both on steering generative protein models towards more complex properties, but mainly on interpreting protein language models as a step towards understanding of the models themselves and applying the techniques from mechanistic interpretability on PLMs.

Bibliography

1. VASWANI, Ashish; SHAZEER, Noam; PARMAR, Niki; USZKOREIT, Jakob; JONES, Llion; GOMEZ, Aidan N.; KAISER, Lukasz; POLOSUKHIN, Illia. *Attention Is All You Need* [online]. arXiv, 2023 [visited on 2026-07-10]. Available from DOI: 10.48550/arXiv.1706.03762. arXiv:1706.03762 [cs.CL].
2. LIN, Zeming; AKIN, Halil; RAO, Roshan; HIE, Brian; ZHU, Zhongkai; LU, Wenting; SMETANIN, Nikita; VERKUIL, Robert; KABELI, Ori; SHMUELI, Yaniv; SANTOS COSTA, Allan dos; FAZEL-ZARANDI, Maryam; SERCU, Tom; CANDIDO, Salvatore; RIVES, Alexander. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* [online]. 2023, vol. 379, no. 6637, pp. 1123–1130 [visited on 2026-07-09]. Available from DOI: 10.1126/science.ade2574.
3. DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton; TOUTANOVA, Kristina. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* [online]. arXiv, 2019 [visited on 2026-07-10]. Available from DOI: 10.48550/arXiv.1810.04805. arXiv:1810.04805 [cs.CL].
4. NIJKAMP, Erik; RUFFOLO, Jeffrey; WEINSTEIN, Eli N.; NAIK, Nikhil; MADANI, Ali. *ProGen2: Exploring the Boundaries of Protein Language Models* [online]. arXiv, 2022 [visited on 2026-07-09]. Available from DOI: 10.48550/arXiv.2206.13517. arXiv:2206.13517 [cs.LG].
5. RADFORD, Alec; WU, Jeffrey; CHILD, Rewon; LUAN, David; AMODEI, Dario; SUTSKEVER, Ilya. Language Models are Unsupervised Multitask Learners. [N.d.].
6. ARORA, Sanjeev; LI, Yuanzhi; LIANG, Yingyu; MA, Tengyu; RISTESKI, Andrej. Linear Algebraic Structure of Word Senses, with Applications to Polysemy. *Transactions of the Association for Computational Linguistics* [online]. 2018, vol. 6, pp. 483–495 [visited on 2026-07-10]. Available from DOI: 10.1162/tacl_a_00034.
7. BRICKEN, Trenton; TEMPLETON, Adly; BATSON, Joshua; CHEN, Brian; JERMYN, Adam; CONERLY, Tom; TURNER, Nick; ANIL, Cem; DENISON, Carson; ASKELL, Amanda; LASENBY, Robert; WU, Yifan; KRAVEC, Shauna; SCHIEFER, Nicholas; MAXWELL, Tim; JOSEPH, Nicholas; HATFIELD-DODDS, Zac; TAMKIN, Alex; NGUYEN, Karina; MCLEAN, Brayden; BURKE, Josiah E; HUME, Tristan; CARTER, Shan; HENIGHAN, Tom; OLAH, Christopher. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. *Transformer Circuits Thread*. 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
8. TEMPLETON, Adly; CONERLY, Tom; MARCUS, Jonathan; LINDSEY, Jack; BRICKEN, Trenton; CHEN, Brian; PEARCE, Adam; CITRO, Craig; AMEISEN, Emmanuel; JONES, Andy; CUNNINGHAM, Hoagy; TURNER, Nicholas L.; MCDUGALL, Callum; MACDIARMID, Monte; TAMKIN, Alex; DURMUS, Esin; HUME, Tristan; MOSCONI, Francesco; FREEMAN, C. Daniel; SUMERS, Theodore R.; REES, Edward; BATSON, Joshua; JERMYN, Adam; CARTER, Shan; OLAH, Chris; HENIGHAN, Tom. *Scaling Monosemanticity: Extract-*

- ing Interpretable Features from Claude 3 Sonnet* [online]. arXiv, 2026 [visited on 2026-07-10]. Available from DOI: 10.48550/arXiv.2605.29358. arXiv:2605.29358 [cs.AI].
9. SIMON, Elana; ZOU, James. InterPLM: discovering interpretable features in protein language models via sparse autoencoders. *Nature Methods*. 2025. Available from DOI: 10.1038/s41592-025-02836-7.
 10. TEMPLETON, Adly; CONERLY, Tom; MARCUS, Jonathan; LINDSEY, Jack; BRICKEN, Trenton; CHEN, Brian; PEARCE, Adam; CITRO, Craig; AMEISEN, Emmanuel; JONES, Andy; CUNNINGHAM, Hoagy; TURNER, Nicholas L; MCDOUGALL, Callum; MACDIARMID, Monte; FREEMAN, C. Daniel; SUMERS, Theodore R.; REES, Edward; BATSON, Joshua; JERMYN, Adam; CARTER, Shan; OLAH, Chris; HENIGHAN, Tom. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread*. 2024. Available also from: <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
 11. THE UNIPROT CONSORTIUM. UniProt: the Universal Protein Knowledgebase in 2025. *Nucleic Acids Research* [online]. 2025, vol. 53, no. D1, pp. D609–D617 [visited on 2026-07-09]. ISSN 1362-4962. Available from DOI: 10.1093/nar/gkae1010.
 12. BERMAN, Helen M.; WESTBROOK, John; FENG, Zukang; GILLILAND, Gary; BHAT, T. N.; WEISSIG, Helge; SHINDYALOV, Ilya N.; BOURNE, Philip E. The Protein Data Bank. *Nucleic Acids Research* [online]. 2000, vol. 28, no. 1, pp. 235–242 [visited on 2026-07-09]. ISSN 0305-1048. Available from DOI: 10.1093/nar/28.1.235.
 13. ALBERTS, Bruce; JOHNSON, Alexander; LEWIS, Julian; RAFF, Martin; ROBERTS, Keith; WALTER, Peter. *Molecular Biology of the Cell*. 4th. Garland Science, 2002. ISBN 978-0-8153-3218-3 978-0-8153-4072-0.
 14. BRANDEN, Carl Ivar; TOOZE, John. *Introduction to Protein Structure*. 2nd ed. New York: Garland Science, 2012. ISBN 978-0-429-06209-4. Available from DOI: 10.1201/9781136969898.
 15. ONUCHIC, José Nelson; WOLYNES, Peter G. Theory of protein folding. *Current Opinion in Structural Biology*. 2004, vol. 14, no. 1, pp. 70–75. ISSN 0959-440X. Available from DOI: 10.1016/j.sbi.2004.01.009.
 16. ANFINSEN, Christian B. Principles that Govern the Folding of Protein Chains. *Science* [online]. 1973, vol. 181, no. 4096, pp. 223–230 [visited on 2026-07-09]. Available from DOI: 10.1126/science.181.4096.223.
 17. JUMPER, John; EVANS, Richard; PRITZEL, Alexander; GREEN, Tim; FIGURNOV, Michael; RONNEBERGER, Olaf; TUNYASUVUNAKOOL, Kathryn; BATES, Russ; ŽÍDEK, Augustin; POTAPENKO, Anna; BRIDGLAND, Alex; MEYER, Clemens; KOHL, Simon A. A.; BALLARD, Andrew J.; COWIE, Andrew; ROMERA-PAREDES, Bernardino; NIKOLOV, Stanislav; JAIN, Rishub; ADLER, Jonas; BACK, Trevor; PETERSEN, Stig; REIMAN, David; CLANCY, Ellen; ZIELINSKI, Michal; STEINEGGER, Martin; PACHOLSKA, Michalina; BERGHAMMER, Tamas; BODENSTEIN, Sebastian; SILVER, David; VINYALS, Oriol; SENIOR, Andrew W.; KAVUKCUOGLU, Koray; KOHLI, Pushmeet; HASABIS, Demis. Highly accurate protein structure prediction with AlphaFold.

- Nature* [online]. 2021, vol. 596, no. 7873, pp. 583–589 [visited on 2026-07-09]. ISSN 0028-0836, ISSN 1476-4687. Available from DOI: 10.1038/s41586-021-03819-2.
18. JAYARAMAN, Vijay; TOLEDO-PATÍÑO, Saacnicteh; NODA-GARCÍA, Lianet; LAURINO, Paola. Mechanisms of protein evolution. *Protein Science : A Publication of the Protein Society* [online]. 2022, vol. 31, no. 7, e4362 [visited on 2026-07-09]. ISSN 0961-8368. Available from DOI: 10.1002/pro.4362.
 19. ROMERO, Philip A.; ARNOLD, Frances H. Exploring protein fitness landscapes by directed evolution. *Nature reviews. Molecular cell biology* [online]. 2009, vol. 10, no. 12, pp. 866–876 [visited on 2026-07-09]. ISSN 1471-0072. Available from DOI: 10.1038/nrm2805.
 20. FINN, Robert D.; CLEMENTS, Jody; EDDY, Sean R. HMMER web server: interactive sequence similarity searching. *Nucleic Acids Research* [online]. 2011, vol. 39, no. Web Server issue, W29–W37 [visited on 2026-07-09]. ISSN 0305-1048. Available from DOI: 10.1093/nar/gkr367.
 21. PRZYTYCKA, Teresa. Lecture 3 Scoring Matrices Position Specific Scoring Matrices Motifs. [N.d.].
 22. TRIVEDI, Rakesh; NAGARAJARAM, Hampapathalu Adimurthy. Intrinsically Disordered Proteins: An Overview. *International Journal of Molecular Sciences* [online]. 2022, vol. 23, no. 22, p. 14050 [visited on 2026-07-09]. ISSN 1422-0067. Available from DOI: 10.3390/ijms232214050.
 23. SUGASE, Kenji; DYSON, H. Jane; WRIGHT, Peter E. Mechanism of coupled folding and binding of an intrinsically disordered protein. *Nature* [online]. 2007, vol. 447, no. 7147, pp. 1021–1025 [visited on 2026-07-09]. ISSN 1476-4687. Available from DOI: 10.1038/nature05858.
 24. ZHANG, Shuai; HE, Zhihui; WANG, Hui; ZHAI, Jingbo. Signal Peptides: From Molecular Mechanisms to Applications in Protein and Vaccine Engineering. *Biomolecules* [online]. 2025, vol. 15, no. 6, p. 897 [visited on 2026-07-09]. ISSN 2218-273X. Available from DOI: 10.3390/biom15060897.
 25. TEUFEL, Felix; ALMAGRO ARMENTEROS, José Juan; JOHANSEN, Alexander Rosenberg; GÍSLASON, Magnús Halldór; PIHL, Silas Irby; TSIRIGOS, Konstantinos D.; WINTHER, Ole; BRUNAK, Søren; HEIJNE, Gunnar von; NIELSEN, Henrik. SignalP 6.0 predicts all five types of signal peptides using protein language models. *Nature Biotechnology* [online]. 2022, vol. 40, no. 7, pp. 1023–1025 [visited on 2026-07-09]. ISSN 1546-1696. Available from DOI: 10.1038/s41587-021-01156-3.
 26. PASSARO, Saro; CORSO, Gabriele; WOHLWEND, Jeremy; REVEIZ, Mateo; THALER, Stephan; SOMNATH, Vignesh Ram; GETZ, Noah; PORTNOI, Tally; ROY, Julien; STARK, Hannes; KWABI-ADDU, David; BEAINI, Dominique; JAAKKOLA, Tommi; BARZILAY, Regina. *Boltz-2: Towards Accurate and Efficient Binding Affinity Prediction* [online]. bioRxiv, 2025 [visited on 2026-07-09]. Available from DOI: 10.1101/2025.06.14.659707. Pages: 2025.06.14.659707 Section: New Results.

27. WATSON, Joseph L.; JUERGENS, David; BENNETT, Nathaniel R.; TRIPPE, Brian L.; YIM, Jason; EISENACH, Helen E.; AHERN, Woody; BORST, Andrew J.; RAGOTTE, Robert J.; MILLES, Lukas F.; WICKY, Basile I. M.; HANIKEL, Nikita; PELLOCK, Samuel J.; COURBET, Alexis; SHEFFLER, William; WANG, Jue; VENKATESH, Preetham; SAPPINGTON, Isaac; TORRES, Susana Vázquez; LAUKO, Anna; DE BORTOLI, Valentin; MATHIEU, Emile; OVCHINNIKOV, Sergey; BARZILAY, Regina; JAAKKOLA, Tommi S.; DIMAIO, Frank; BAEK, Minkyung; BAKER, David. De novo design of protein structure and function with RFdiffusion. *Nature* [online]. 2023, vol. 620, no. 7976, pp. 1089–1100 [visited on 2026-07-09]. ISSN 1476-4687. Available from DOI: 10.1038/s41586-023-06415-8.
28. STEINEGGER, Martin; SÖDING, Johannes. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology* [online]. 2017, vol. 35, no. 11, pp. 1026–1028 [visited on 2026-07-09]. ISSN 1546-1696. Available from DOI: 10.1038/nbt.3988.
29. BOUTET, Emmanuel; LIEBERHERR, Damien; TOGNOLLI, Michael; SCHNEIDER, Michel; BAIROCH, Amos. UniProtKB/Swiss-Prot. *Methods in Molecular Biology*. 2007, vol. 406, pp. 89–112. ISSN 1064-3745. Available from DOI: 10.1007/978-1-59745-535-0_4.
30. SIGRIST, Christian J. A.; CUCHE, Béatrice A.; CASTRO, Edouard de; COUDERT, Elisabeth; REDASCHI, Nicole; BRIDGE, Alan. The PROSITE database for protein families, domains, and sites. *Nucleic Acids Research*. 2026, vol. 54, no. D1, pp. D451–D458. ISSN 1362-4962. Available from DOI: 10.1093/nar/gkaf1188.
31. ALTSCHUL, S. F.; GISH, W.; MILLER, W.; MYERS, E. W.; LIPMAN, D. J. Basic local alignment search tool. *Journal of Molecular Biology*. 1990, vol. 215, no. 3, pp. 403–410. ISSN 0022-2836. Available from DOI: 10.1016/S0022-2836(05)80360-2.
32. ARMENTEROS, Jose Juan Almagro; SALVATORE, Marco; EMANUELSSON, Olof; WINTHER, Ole; HEIJNE, Gunnar von; ELOFSSON, Arne; NIELSEN, Henrik. Detecting sequence signals in targeting peptides using deep learning. *Life Science Alliance* [online]. 2019, vol. 2, no. 5 [visited on 2026-07-09]. ISSN 2575-1077. Available from DOI: 10.26508/lsa.201900429.
33. MEHDIABADI, Mahta; BLUM, Matthias; TESEI, Giulio; BÜLOW, Sören von; LINDORFF-LARSEN, Kresten; TOSATTO, Silvio C E; PIOVESAN, Damiano. MobiDB-lite 4.0: faster prediction of intrinsic protein disorder and structural compactness. *Bioinformatics* [online]. 2025, vol. 41, no. 5, btaf297 [visited on 2026-07-09]. ISSN 1367-4811. Available from DOI: 10.1093/bioinformatics/btaf297.
34. SU, Jianlin; LU, Yu; PAN, Shengfeng; MURTADHA, Ahmed; WEN, Bo; LIU, Yunfeng. RoFormer: Enhanced Transformer with Rotary Position Embedding [online]. arXiv, 2023 [visited on 2026-07-11]. Available from DOI: 10.48550/arXiv.2104.09864. arXiv:2104.09864 [cs.CL].

35. ELHAGE, Nelson; NANDA, Neel; OLSSON, Catherine; HENIGHAN, Tom; JOSEPH, Nicholas; MANN, Ben; ASKELL, Amanda; BAI, Yuntao; CHEN, Anna; CONERLY, Tom; DASSARMA, Nova; DRAIN, Dawn; GANGULI, Deep; HATFIELD-DODDS, Zac; HERNANDEZ, Danny; JONES, Andy; KERNION, Jackson; LOVITT, Liane; NDOUSSE, Kamal; AMODEI, Dario; BROWN, Tom; CLARK, Jack; KAPLAN, Jared; MCCANDLISH, Sam; OLAH, Chris. A Mathematical Framework for Transformer Circuits. *Transformer Circuits Thread*. 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
36. SAPHRA, Naomi; WIEGREFFE, Sarah. *Mechanistic?* [online]. arXiv, 2024 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2410.09087. arXiv:2410.09087 [cs.AI].
37. NOSTALGEBRAIST. *interpreting GPT: the logit lens* [online]. 2020. [visited on 2026-07-12]. Available from: <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>.
38. ALAIN, Guillaume; BENGIO, Yoshua. *Understanding intermediate layers using linear classifier probes* [online]. arXiv, 2018 [visited on 2026-07-11]. Available from DOI: 10.48550/arXiv.1610.01644. arXiv:1610.01644 [stat.ML].
39. BELINKOV, Yonatan. Probing Classifiers: Promises, Shortcomings, and Advances. *Computational Linguistics* [online]. 2022, vol. 48, no. 1, pp. 207–219 [visited on 2026-07-12]. Available from DOI: 10.1162/coli_a_00422.
40. MIKOLOV, Tomas; CHEN, Kai; CORRADO, Greg; DEAN, Jeffrey. *Efficient Estimation of Word Representations in Vector Space* [online]. arXiv, 2013 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.1301.3781. arXiv:1301.3781 [cs.CL].
41. PARK, Kiho; CHOE, Yo Joong; VEITCH, Victor. *The Linear Representation Hypothesis and the Geometry of Large Language Models* [online]. arXiv, 2024 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2311.03658. arXiv:2311.03658 [cs.CL].
42. ELHAGE, Nelson; HUME, Tristan; OLSSON, Catherine; SCHIEFER, Nicholas; HENIGHAN, Tom; KRAVEC, Shauna; HATFIELD-DODDS, Zac; LASENBY, Robert; DRAIN, Dawn; CHEN, Carol; GROSSE, Roger; MCCANDLISH, Sam; KAPLAN, Jared; AMODEI, Dario; WATTENBERG, Martin; OLAH, Christopher. Toy Models of Superposition. *Transformer Circuits Thread*. 2022. https://transformer-circuits.pub/2022/toy_model/index.html.
43. GAO, Leo; TOUR, Tom Dupré la; TILLMAN, Henk; GOH, Gabriel; TROLL, Rajan; RADFORD, Alec; SUTSKEVER, Ilya; LEIKE, Jan; WU, Jeffrey. *Scaling and evaluating sparse autoencoders* [online]. arXiv, 2024 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2406.04093. arXiv:2406.04093 [cs.LG].
44. BUSSMANN, Bart; LEASK, Patrick; NANDA, Neel. *BatchTopK Sparse Autoencoders* [online]. arXiv, 2024 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2412.06410. arXiv:2412.06410 [cs.LG].

45. TURNER, Alexander Matt; THIERGART, Lisa; LEECH, Gavin; UDELL, David; VAZQUEZ, Juan J.; MINI, Ulisse; MACDIARMID, Monte. *Steering Language Models With Activation Engineering* [online]. arXiv, 2024 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2308.10248. arXiv:2308.10248 [cs.CL].
46. *PLM-eXplain: divide and conquer the protein embedding space / Bioinformatics / Oxford Academic* [online]. [N.d.]. [visited on 2026-07-12]. Available from: <https://academic.oup.com/bioinformatics/article/42/1/btaf631/8339745>.
47. LIU, Xiangyu; LEI, Haodi; LIU, Yi; LIU, Yang; HU, Wei. *ProtSAE: Disentangling and Interpreting Protein Language Models via Semantically-Guided Sparse Autoencoders* [online]. arXiv, 2026 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2509.05309. arXiv:2509.05309 [q-bio.QM].
48. ADAMS, Etowah; BAI, Liam; LEE, Minji; YU, Yiyang; ALQURAISHI, Mohammed. *From Mechanistic Interpretability to Mechanistic Biology: Training, Evaluating, and Interpreting Sparse Autoencoders on Protein Language Models*. bioRxiv: The Preprint Server for Biology, 2025. Available from DOI: 10.1101/2025.02.06.636901. ISSN: 2692-8205.
49. HUANG, Long-Kai; ZHU, Rongyi; HE, Bing; YAO, Jianhua. *Steering Protein Language Models* [online]. arXiv, 2025 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2509.07983. arXiv:2509.07983 [q-bio.BM].
50. GARCIA, Edith Natalia Villegas; ANSUINI, Alessio. *Interpreting and Steering Protein Language Models through Sparse Autoencoders* [online]. arXiv, 2025 [visited on 2026-07-12]. Available from DOI: 10.48550/arXiv.2502.09135. arXiv:2502.09135 [cs.LG].
51. COROMINAS, Gerard Boxo; STOCCO, Filippo; FERRUZ, Noelia. Sparse Autoencoders in Protein Engineering Campaigns: Steering and Model Diffing. In: [online]. 2025 [visited on 2026-07-12]. Available from: <https://openreview.net/forum?id=rnJ6Nn1Wf5>.
52. SAMUEL MARKS, Adam Karvonen; MUELLER, Aaron. *dictionary_earning* [https://github.com/saprmarks/dictionary_learning]. 2024.
53. *Streamlit • A faster way to build and share data apps* [online]. 2021. [visited on 2026-07-12]. Available from: <https://streamlit.io/>.
54. KEMPEN, Michel van; KIM, Stephanie S.; TUMESCHEIT, Charlotte; MIRDITA, Milot; LEE, Jeongjae; GILCHRIST, Cameron L. M.; SÖDING, Johannes; STEINEGGER, Martin. Fast and accurate protein structure search with Foldseek. *Nature Biotechnology* [online]. 2024, vol. 42, no. 2, pp. 243–246 [visited on 2026-07-14]. ISSN 1546-1696. Available from DOI: 10.1038/s41587-023-01773-0.

List of Figures

1.1	Table of standard 20 amino acids, including the one- and three-letter codes, grouped by physicochemical properties. Retrieved on 29.6.2026 from https://cdn.technologynetworks.com/tn/images/body/aminoacids-pic3revised1574260662291.png . . .	10
1.2	A. Chain of amino acids forming peptide bonds. retrieved on 9.7.2026 from https://www.ebi.ac.uk/training/online/courses/biomacromolecular-structures/proteins/levels-of-protein-structure-primary/ B. Secondary structure elements, alpha helix and beta sheet. retrieved on 9.7.2026 from https://www.mun.ca/biology/scarr/MGA2_03-18b.html C. Ternary structure of a P-loop protein (PDB: 6C2U). D. Quaternary structure of ammonia lyase (PDB 1JSW), colored by chain, screenshots from PyMOL.	11
2.1	Architecture of the transformer model [1], showing both the encoder and decoder	20
3.1	Schema of the interpretability pipeline, including feature analysis modes through command-line interface (CLI) and interactive dashboard. SAE diagram retrieved on 15.7.2026 from https://adamkarvonen.github.io/machine_learning/2024/06/11/sae-intuitions.html . PLM embedder is used for different purposes: first for generating training data for SAEs, and second for embedding annotated sequences used for analysis, as marked by the red and yellow-colored arrows. The remaining arrow colors have no meaning.	28
3.2	Screenshots from the dashboard. A: Feature explorer. B: Concept explorer. C: Protein zoom. D: Feature finder.	39
4.1	Per-concept-type detection coverage for nine of the trained SAEs. Each cell gives the number of concepts of that type identified at test $F_1 \geq 0.5$, shaded by the fraction of the type covered. Columns are grouped by model, with the ESM-2 SAEs from InterPLM on the left and our ProGen2 SAEs on the right.	45
4.2	Activation profiles of the top 5 features by enrichment for the ATP binding site concept. Computed for sequence with UniProt ID: Q0HYN8	48
4.3	Protein zoom view for a C2H2 zinc-finger protein. Green regions mark the annotated zinc-finger domains, and the stacked bar graphs give the activation of each of the five features from Table 4.5 across the sequence.	50
4.4	Achieved percentage of disordered residues for various features and clamp values. Top two plots use k=10 SAE and bottom two use k=350 SAE. The grid over which clamp values are tested is different for each SAE, both starting with the unsteered baseline.	53

- 4.5 MobiDB-lite compositional bias breakdown for disorder-associated SAE features at fixed clamps (k10 @ $3\times$, k350 @ $8\times$), showing the fraction of generated residues attributed to each bias sub-type (stacked bars) alongside total disorder fraction (black diamonds); the dashed line marks the unsteered baseline. Features are sorted by disorder fraction; italic labels indicate F1/dom-selected features. 54

List of Tables

4.1	Training hyperparameters for the ProGen2 SAEs we trained (the ESM-2 SAEs we compare against are pretrained from InterPLM).	42
4.2	Sparse autoencoders trained in this work: training hyperparameters, reconstruction fidelity, and feature-level interpretability metrics.	44
4.3	Top five features for the ATP binding site concept, ranked by annotation enrichment score, on the ProGen2-large layer-24 $k=350$ SAE. <i>Enr</i> is the enrichment ratio, <i>F1d</i> is the test F_1 -per-domain, and <i>Prec</i> is the precision. Random baselines: hit@1 = 0.02, hit@5 = 0.09, AUPRC = 0.024.	46
4.4	Top five features for the nucleophile active site, ranked by annotation enrichment, on the ProGen2-large layer-24 $k=350$ SAE. Columns as in Table 4.3. Random baselines: hit@1 = 0.003, hit@5 = 0.014, AUPRC = 0.003.	47
4.5	Top five features for the C2H2-type zinc finger, ranked by F_1 -per-domain, on the ProGen2-large layer-24 $k=10$ SAE. Columns as in Table 4.3, without the enrichment ratio. Retrieval columns are averaged over the 108 labeled proteins. Random baselines: hit@1 = 0.231, hit@5 = 0.732, AUPRC = 0.318.	49
4.6	Retrieval metrics and steerability for all tested disorder features across both SAE checkpoints ($k=10$ and $k=350$). Rk_{F1} and Rk_{F1d} are global ranks by F_1 and F_1 -per-domain across all features for Region_Disordered concept. Within each SAE block, the upper group was selected by F_1 and the lower by $F1d$.	52
4.7	Generation results ($n = 128$ per condition, prefix 1M). Dis%(r) = mean fraction of residues in consensus disorder; Dis%(s) = fraction of sequences with any disorder call; NPoly% = MobiDB-lite compositional bias denoting acidic residues; D+E% and P% = mean residue-level amino acid composition (manually computed); UAA = mean unique amino acids per sequence; MaxRun = mean longest consecutive run of one amino acid.	55
4.8	Generation after a prompt, with SAE feature steering ($n=128$). Each condition shows the baseline and steered value for its target metric only: D+E% for Acidic, P% for Pro, disorder fraction for Disorder. k10 clamp 3 $\times$, k350 clamp 8 $\times$. † CheY baseline triggers early EOS after the 129-aa prefix; steered runs override termination.	56
4.9	Target peptide steering configurations grouped into single features, combinations, and suppression. Within the combinations block the indented rows build on the three-feature positive combination above them. %TP is the percentage of sequences predicted as TP ($n=512$ per experiment) %frag and NLL (negative log likelihood) are sequence-quality proxies.	59

A Attachments

A.1 Code

Code for this thesis is in a ZIP file submitted as attachment to this thesis. However, we highly recommend to look at the GitHub <https://github.com/hugohrban/InterPLM/> instead. The repo is a fork of <https://github.com/ElanaPearl/InterPLM/>. The commits made by me on top of the original repo are in the git tree, the attached ZIP file also contains the `.git/` folder.

A.2 Changelog

Covers commits authored by me on top of the upstream `InterPLM`. Changes are ordered by significance: major features, then bug fixes, then usability/convenience improvements.

Major New Features

- **ProGen2 support end-to-end:** new ProGen2 embedder registered as a PLM backend alongside ESM-2 (`interplm/embedders/progen2.py`, `interplm/embedders/__init__.py`), plus ProGen-specific fidelity computation and activation intervention (`interplm/train/fidelity.py`, `interplm/sae/intervention.py`, `interplm/analysis/concepts/extract_annotations.py`, `scripts/evaluate_sae.py`), and support for the large ProGen2 checkpoint (`interplm/embedders/progen2.py`, `scripts/run_concept_analysis.py`).
- **Vectorized concept metrics + parallelized annotation extraction:** replaced per-concept loops with a single matmul for metric calculation and switched shard processing to `ProcessPoolExecutor` (`interplm/analysis/concepts/compare_activations.py`, `interplm/analysis/concepts/extract_annotations.py`, `interplm/embedders/esm.py`). Took the concept-analysis pipeline from ~24 hours to ~20 minutes in practice, the single biggest practical speedup in the fork.
- **Multi-GPU, resumable embedding/annotation extraction pipeline:** sharded work across all available GPUs via `ProcessPoolExecutor`, auto-halved batch size on OOM and retried, skipped shards already processed, and kept the embedder resident in worker processes instead of reloading per shard (`scripts/extract_embeddings.py`, `scripts/embed_annotations.py`). Also added single-pass multi-layer extraction instead of one forward pass per layer (`scripts/embed_annotations.py`, `interplm/embedders/esm.py`, `interplm/embedders/progen2.py`).
- **CLI-driven SAE training** (`scripts/train_sae.py`): trains any of the four SAE variants (`relu/topk/batch_topk/jump_relu`) via `tapify`, with a

-dry_run profiling mode and auto-generated hyperparameter-encoded save directories.

- **Rank-based concept evaluation framework:** scores SAE features against a biological concept via F1-guided ranking or annotation-enrichment (`interplm/analysis/concepts/rank_eval.py`, `scripts/quick_rank_eval.py`, `scripts/rank_eval_concepts.py`), with a precomputed sufficient-statistics cache so repeat concept queries are instant instead of rerunning a full GPU pass (`interplm/analysis/concepts/rank_eval.py`, `scripts/rank_eval_concepts.py`).
- **Concept Explorer & Protein Zoom dashboard modes:** a searchable dashboard view of SAE features by biological concept, backed by rank eval results (`interplm/dashboard/concept_explorer.py`, `interplm/dashboard/app.py`, `interplm/dashboard/dashboard_cache.py`, `interplm/dashboard/feature_activation_vis.py`, `scripts/create_dashboard.py`), plus a “Protein Zoom” page comparing per-residue activation profiles of chosen features on a chosen protein, with cross-navigation between dashboard pages (`interplm/dashboard/protein_zoom.py`, `interplm/dashboard/app.py`, `interplm/dashboard/concept_explorer.py`, `interplm/dashboard/feature_activation_vis.py`, `interplm/analysis/per_protein_analysis.py`).
- **Per-protein SAE feature analysis module:** identifies which features fire in a given protein, classifies them by cross-dataset specificity and within-protein activation pattern, and reports concept-annotation overlap, with a lazy shard index for fast lookup over large embedding sets (`interplm/analysis/per_protein_analysis.py`, `scripts/protein_feature_analysis.py`).
- **Activation steering for SAE feature intervention:** hooks a PLM layer during generation, edits an SAE feature (clamp/ablate), and decodes back (direct or with-error reconstruction); later extended with multi-feature steering, batched generation, hook verification, KV-cache generation, and prefix-skip steering so only newly generated tokens are steered (`scripts/activation_steering_naive.py`, `.claude/skills/steer-protein/SKILL.md`).
- **Anti-target feature discovery + steering evaluation tooling:** finds SAE features enriched in negative vs. positive examples (e.g. no-transit-peptide vs. transit-peptide sequences) using TargetP2, ps_scan, or SignalP6 as label sources (`scripts/find_antitarget_features.py`), plus MobiDB-lite-based evaluation of steered outputs: disorder fraction, compositional bias, sequence diversity stats (`scripts/eval_mobidb_steering.py`), and a written skill guide for the steering workflow (`.claude/skills/steer-protein/SKILL.md`).
- **ESM-2 vs. ProGen2 comparison research tooling:** systematic concept-coverage comparison: precision/recall/F1/AUROC across check-

points (`scripts/compare_esm_progen.py`), and a positional analysis testing whether causal attention anchors N-terminal features more sharply at sequence start (`scripts/positional_analysis.py`).

- **Consolidated concept-analysis metrics & reporting:** in-sample validation metrics (best F1 per concept without cross-split selection) and a richer report: F1 distribution, valid→test F1 gap, concept coverage, polysemanticity, Spearman rank stability, JSON summary (`interplm/analysis/concepts/report_metrics.py`, `scripts/run_concept_analysis.py`), plus aggregation of per-SAE metrics into one consolidated results CSV (`scripts/run_concept_analysis.py`).
- **Dead-neuron resampling on by default + full-batch fidelity checks:** changed SAE training defaults to resample dead neurons and run fidelity evaluation across all batches after training instead of a subset (`interplm/train/training_run.py`, `scripts/train_sae.py`).

Bug Fixes

- Fixed categorical annotation parsing to use the correct per-feature separator field: `Binding site` and `Cofactor` columns were silently zeroed out because the parser hardcoded `/note=` when those feature types actually use `/ligand=` and `/Name=` (`interplm/analysis/concepts/extract_annotations.py`, `interplm/analysis/concepts/parsing_utils.py`).
- Fixed a threshold-count mismatch in F1 calculation (hardcoded 10 thresholds vs. the actual 5 used when generating shard counts), which caused an out-of-bounds index (`interplm/analysis/concepts/calculate_f1.py`, `interplm/analysis/concepts/compare_activations.py`).
- Fixed SAE features not being normalized before thresholding in concept activation comparison (`interplm/analysis/concepts/compare_activations.py`).
- Fixed a dtype cast bug in dead-neuron resampling weights (`interplm/train/trainers/relu.py`).
- Fixed fidelity batch sampling to use a batch count instead of a training-step interval, and a related numpy-array slicing `TypeError` (`interplm/train/fidelity.py`).
- Reverted an accidental regression where note normalization had become dead code, restoring trailing-instance-index stripping (e.g. “Sushi 1” → “Sushi”) (`interplm/analysis/concepts/parsing_utils.py`).
- Fixed the AFDB structure download to use the current prediction API and model v6 after the old endpoint broke (`interplm/dashboard/view_structures.py`).
- Fixed a broken ProGen2 model reference by prefixing it with the `hugohrban/HF` namespace (`scripts/protein_feature_analysis.py`).

- Fixed Streamlit warnings and broken walkthrough/tutorial steps (`interplm/dashboard/app.py`, `interplm/sae/intervention.py`, `interplm/train/fidelity.py`).

Usability / Convenience Improvements

- Replaced noisy per-shard progress bars with a single unified bar (`interplm/analysis/per_protein_tracking.py`).
- Made the dashboard load its embedder lazily on first use instead of at startup (`interplm/dashboard/app.py`).
- Auto-discover all shards in an embeddings directory when none are explicitly specified (`scripts/collect_feature_activations.py`).
- Auto-detect the SAE class from checkpoint keys instead of requiring it to be specified (`scripts/create_dashboard.py`).
- `evaluate_sae` now accepts a directory of FASTA files (not just a single file); fidelity parses FASTA format directly via BioPython (`scripts/evaluate_sae.py`, `interplm/train/fidelity.py`).
- Added `force_include_patterns` to guarantee specific annotation types (e.g. rare domains) bypass min-count thresholds and are always included in eval subsets / annotation extraction (`interplm/analysis/concepts/prepare_eval_set.py`, `interplm/analysis/concepts/extract_annotations.py`).
- Normalized trailing instance indices in UniProtKB domain notes so repeated instances of a domain are grouped together (`interplm/analysis/concepts/parsing_utils.py`).
- Centralized default threshold percentages into a single shared constant instead of duplicating the list across callers (`interplm/analysis/concepts/concept_constants.py`, `interplm/analysis/concepts/calculate_f1.py`, `interplm/analysis/concepts/compare_activations.py`).
- Increased the dead-feature threshold to 20M tokens (fewer false-positive “dead” features on longer runs) (`interplm/train/trainers/batch_top_k.py`).
- Added a `verbose` flag to suppress per-token progress bars during batched steering generation (`scripts/activation_steering_naive.py`).
- Cleaned up progress reporting and debug noise: `tqdm` progress and silenced debug prints (`interplm/sae/normalize.py`, `interplm/train/configs.py`).

A.3 Models

Trained sparse autoencoders for ProGen2 large, layer 24, along with the dashboard cache data are stored on the Hugging face platform: https://huggingface.co/hugohrban/progen2_large_L24_SAE_k10 and https://huggingface.co/hugohrban/progen2_large_L24_SAE_k350, which are the two main SAEs used throughout this work. The `README.md` files in Hugging face contain the instructions for launching the dashboard.